

Developing and Characterizing a Maximum Entropy Model to Analyze Memory and Prediction in Cultured Cortical Neurons

Chris Wang

Professor Sarah Marzen (Keck), Advisor

Professor Jorge Moreno (POM), Advisor

Professor Thomas Moore (POM), Advisor



A thesis submitted in partial fulfillment
of the requirements for the
Degree of Bachelor of Arts
in Physics and Astronomy

POMONA COLLEGE

Claremont, CA

May 7, 2025

Abstract

This thesis develops and characterizes a Maximum Entropy (MaxEnt) model for analyzing memory and prediction in cultured cortical neurons. Using principles from statistical mechanics and information theory, I constructed a model that infers correlations between neurons by minimizing Kullback-Leibler divergence. The model was validated against synthetic data, showing the expected $n^{-1/2}$ convergence characteristic of Maximum Likelihood Estimation. When applied to real neuronal data from both global and local stimulation experiments with zero timeshift, the model outperformed baseline predictions, achieving 98.34% accuracy for inactive neurons and 38.39% accuracy for active neurons in global stimulation cases. In the local case, the model achieved 99.66% predictive accuracy for inactive neurons and 67.01% for active neurons. Results suggest that neural networks effectively store representations of their present environment and maintain short-term memory (up to 100ms), but show limited predictive capabilities for future states. Analysis reveals that strategic selection of neuron subsets significantly improves model performance while maintaining computational efficiency.

Acknowledgments

First and foremost, I would like to thank Professor Sarah Marzen for graciously taking me under her wing about eight months ago. Without her serious intellect and kind soul, this thesis could not be complete, nor would I have solidified my interest in researching complex systems through the lens of mathematical physics. There were many periods throughout the last eight months where I could not be at my best, but each time, Prof. Marzen wholeheartedly understood my situation and sent me well wishes. She has also helped me grow as a scientist, pushing me to present at an academic conference for my first time this past March and broadening my experience as a physicist.

I would also like to thank Professor Thomas Moore for being my former research advisor and significantly impactful figure in my academic growth. Ever since I first spoke to him at alumni weekend in 2023, he has continuously enlightened me with his many wisdoms. I will never forget the two-hour conversation we had in his office, where I initially came in to ask about the history of 20th century physics and left with not only that, but also a better understanding of Hebrew (in particular, it is less noun-focused than English) and anecdotes about family members.

I would also like to thank Professors Dwight Whitaker and Philip Choi for their guidance throughout my college years. Having taken Physics 70 with Dwight and Astro 51 with Prof. Choi, I have known them for the longest out of any faculty member here. Whenever I have a problem about course selection or want to share how I've been feeling about recent events or the future, they both welcome me with an open door. I am excited to see where we will all be in years, even decades. Additionally, I would like to thank Profe Jorge Moreno for reading my thesis on such short notice and providing detailed feedback on both the written and spoken

portions of it.

Of course, I would also like to thank my friends for getting me to the finish line. Thank you to Oscar, Tom, and Larry for always being a beacon of lightheartedness and overall joy in the past few years. I appreciated the many conversations and whale noises we exchanged and how our group has stayed tight even though Oscar is now in the UK. Here's to hoping we continue floating through the oceans of life like sea otters. Thank you to my underclassmen physics friends who have brought me much joy in my ultimate year as a physics major — Shawn, Eve, Atharv, Jacob, Ava, Nora, to name a few. Don't forget about $\langle A|A \rangle$. Thank you to Claire and Ethan for befriending me and staying close throughout the whole time. My first year wasn't easy, but you both stuck with me through it all and made me feel valued even when it was hard. I have enjoyed every conversation we have had, and they have poked my brain to think about things in intriguing ways. Thank you to Anbo for an awesome time this and last year; I loved both the serious and unserious conversations we've had in our suite hallway. Thank you to Joey and Haram for being honorary suitemates of Dialynas 100.

Finally, I am only here thanks to the support of my family.

妈妈爸爸，这些年来我们一家都经历了很多，也都成长了不少。大学第一学期回家时跟你们的谈话让我真正体会到，原来父母和孩子之间的爱可以有很多种表达方式。你们不仅认真听我分享经历，还在听完之后调整了表达爱的方式，让我感受到一种无比温暖的力量。

这种温暖就像一面盾牌，无论外界如何变化，它都保护着我，引导我成长为一个懂得爱自己的人。你们对我的耐心，以及对我能力的信任，让我的自信像竹子一样稳稳地从地里长出来。

虽然你们有时会批评我，但现在我明白，那是出于爱和我未来的关心。我也希望你们能感受到我对你们的爱和敬佩。每当我在生活中遇到问题，我总会第一时间想到你们，向你们寻求帮助。

爸爸妈妈，我爱你们。

Contents

Abstract	i
Acknowledgments	ii
1 Introduction	1
2 Theoretical Foundations	4
2.1 An Example to Contextualize Entropy	4
2.2 Shannon-Flavored Entropy	6
2.3 The Kullback-Leibler Divergence	8
2.4 Bringing Entropy and Divergence Together	10
3 Constructing and Validating the Model	13
3.1 Maximum Likelihood Estimation (MLE)	14
3.1.1 Before I Dive In: Why MLE?	15
3.2 Testing the Model	16
3.2.1 The Model is Asymptotically Unbiased	16
4 Confusion Matrices: Classifying the Categorical	19
5 What Happens in Real Neuron Populations?	24
5.1 Real Data: Cleaning and Processing, with some Modifications	24
5.2 An Initial Survey (and Rationalization) of the Data	27
5.3 Global Stimulation	29
5.4 Local Stimulation	31

6	Key Takeaways and Future Work	35
A	Calculations for the Maximum Entropy Model	37
B	Non-Timeshifted, MaxEnt-inferred $\mathcal{J}$ Matrices for Experiments and Corresponding Confusion Matrices	41
B.1	$\mathcal{J}$ Matrices For Global/Optogenetic Stimulation	42
B.2	$\mathcal{J}$ Matrices For Local/Focal stimulation	43
B.3	Confusion Matrices For Global/Optogenetic Stimulation	44
B.4	Confusion Matrices For Local/Focal Stimulation	47

List of Figures

3.1	Frobenius norm plot of $\mathcal{J}$ matrices as a function of data size	18
4.1	Test Data Visualization	19
4.2	Confusion matrices for well-behaved data	22
5.2	Confusion matrices for global stimulation, no timeshift	30
5.3	Global stimulation predictive accuracy	31
5.4	Confusion matrices for local stimulation, no timeshift	32
5.5	Local stimulation predictive accuracy	33
B.1	Confusion Matrices, Experiment 1 and 2 global	44
B.2	Confusion Matrices, Experiment 3 and 4 global	45
B.3	Confusion Matrices, Experiment 5 and 6 global	45
B.4	Confusion Matrices, Experiment 7 and 8 global	46
B.5	Confusion Matrices, Experiment 9 and 10 global	46
B.6	Confusion Matrices, Experiment 9, 12, 14, and 15 local	47
B.7	Confusion Matrices, Experiment 16 and 17 local	48
B.8	Confusion Matrices, Experiment 18 and 19 local	48

Chapter 1

Introduction

Biophysics, particularly the study of biological complex systems, represents a relatively new application of physical principles. While physics has elucidated our understanding of the sky above and the subatomic particles within, its methodologies have only recently been adapted to decode the intricacies within living systems. Statistical physics, with its powerful conceptual toolkit and mathematical foundations, has proven fruitful in this domain. Concepts like entropy, which originated in thermodynamics to describe the ordered-ness of a system, now help us understand information processing in cells. The frameworks of microstates and macrostates, originally developed to describe gas molecules, provide insights into protein folding and neuron activation. Even the canonical ensemble, a mathematical framework describing systems in thermal equilibrium, finds unexpected applications in modeling genetic networks and neural activity patterns.

The significance of this cross-disciplinary approach cannot be overstated. Biology, once considered too messy and complex for rigorous physical analysis, increasingly yields to these adapted formalisms, with neural networks forming a prime example. Insights from statistical mechanics have not only illuminated brain function but have spilled over into technological innovation: the 2024 Nobel Prize in Physics was awarded to John Hopfield and Geoffrey Hinton for applying statistical mechanics to create artificial networks capable of storing memories. Such revolutionary work fundamentally underpins today's artificial intelligence revolution.

The intellectual foundations of our approach trace back to a specific philosophy. In 1957, E.T.

Jaynes introduced what would come to be known as the Maximum Entropy (MaxEnt) model [1, 2] demonstrating that statistical mechanics could be derived as a specific application of information theory. This was paradigm-shifting: it suggested that the physical laws we observe emerge from optimal inference given limited information rather than from underlying mechanical necessity alone.

Nearly five decades later, this conceptual bridge between information and physics found powerful application in neuroscience. In 2006, Schneidmann et al. published a landmark paper in *Nature* that applied the MaxEnt approach to neural networks [3, 4]. Their crucial insight was that by assuming only first-order pairwise correlations between neurons, the resulting mathematical model was equivalent to the Ising model from statistical physics. The Ising model consists of a lattice of binary elements that interact with their neighbors and was originally developed to study ferromagnetism. The collective behavior of these simple binary units creates complex patterns across the system, much like neurons that either fire or remain silent while influencing how likely neighboring cells fire.

In the following chapters, I explore the application of maximum entropy models to understand memory and prediction within networks of cultured cortical neurons. At its core, this investigation addresses how biological organisms form impressions of their environment. The basic assumption is that any organism with a biological sensor interacts with their environment through various mechanisms: creating noisy representations of present conditions, storing memories of past states, or generating predictions of future events.

The neural networks under examination provide an ideal system for studying these processes. At every given moment, each neuron in the network is either firing (on) or silent (off). Using the maximum entropy framework, I attempt to predict the state of a specific neuron, our “sensor”, based on the activity patterns of other neurons in the network, which collectively represent the “environment.” This approach mirrors how biological systems extract meaningful signals from complex surroundings.

The technical heart of this work involves developing an estimator based on maximum entropy

principles and rigorously analyzing its performance. This analysis reveals the nature and strength of correlations between the environmental neurons and the biological sensor. By quantifying how well we can predict the sensor's state from environmental information, we better understand fundamental questions about information processing in neural systems. How much environmental information can a single neuron capture? What structural or temporal patterns in neural activity enable effective prediction? How do these mechanisms relate to the organism's ability to represent its world?

To guide the reader through this investigation, I offer a brief roadmap of what is to come. Chapter 2 establishes the theoretical framework, exploring the conceptual foundations and fleshing out the mathematics of maximum entropy, showing how information-theoretic principles can be applied to neural systems without imposing unnecessary assumptions. Chapter 3 transitions into implementation, documenting the computational methods used to create and validate our models. This includes the algorithms developed to handle the high-dimensional data spaces characteristic of neural networks and verification procedures to ensure mathematical consistency.

In Chapter 4, I present an estimator I developed, derived from our maximum entropy model. This chapter characterizes the performance of this estimator using analytical tools that quantify its predictive power and is tested on well-behaved data, which acts as another layer of model validation. Chapter 5 applies these methods to real neuronal data, presenting experimental results and their interpretation within our theoretical framework. I expound on the predictive accuracy of our maximum entropy model on real data, comparing it to a baseline predictor to give us a sense of how much information the model can capture about the memory and predictive capabilities of such networks.

Chapter 2

Theoretical Foundations

2.1 An Example to Contextualize Entropy

In statistical mechanics and information theory, the concept of *entropy* is of great importance. When Ludwig Boltzmann introduced the concept of entropy in his exploration of statistical mechanics, he wrote it as

$$S = k_B \ln \Omega, \tag{2.1}$$

with S denoting entropy, k_B a scaling constant known as Boltzmann's constant, and Ω the number of **microstates**, or possible configurations, of a particular physical system. Within that physical system, though, we can also quantify its larger-scale properties, which is known as a system's **macrostate**. For instance, if I roll two fair six-sided dice, configurations in which the dice sum up to seven is a macrostate of the system. It turns out that there are six microstates in that specific macrostate, which you can verify yourself. Naturally, one can see that the more *microstates* a system has (or a particular configuration of the system), the higher its entropy. At its core, entropy quantifies the number of configurations available in a physical system. It is convenient to use the natural log due to its ability to simplify multiplication into addition.

Why do we care about using entropy to model systems? To motivate this, let us proceed by example. Consider a row of ten coins. I flip each one once and take a note of whether it lands on heads or tails. If a coin lands on heads, I tally a 1; if it lands on tails, I tally a 0. If I make a list

for each result, I obtain a 10-dimensional binary vector, with an example shown here:

$$\mathbf{x} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

The above example depicts the result when the first five coins land heads up and the last five coins land tails up. Now I ask two questions: how many possible states are there for this system to adopt, and in how many of them do I obtain k heads and $10 - k$ tails? The first question is relatively straightforward: each entry can adopt one of two states, so for this 10-dimensional binary vector, I can get a total of $2^{10} = 1024$ states. More generally, for an N -dimensional binary vector, I obtain a total of 2^N states. To address the second question, we take a tool from combinatorics: the choose function. Specifically, the number of ways one can choose k items from a population of N items is given by

$$\binom{N}{k} = \frac{N!}{k!(N-k)!}. \quad (2.2)$$

Applying that to our case, we see that the number of states in which we get k heads is 10 choose k . I will also denote this quantity as $\Omega_{k,10-k}$ for compactness, and the table below illustrates each case.

$\binom{10}{0}$	$\Omega_{0,10}$	1
$\binom{10}{1}$	$\Omega_{1,9}$	10
$\binom{10}{2}$	$\Omega_{2,8}$	45
$\binom{10}{3}$	$\Omega_{3,7}$	120
$\binom{10}{4}$	$\Omega_{4,6}$	210
$\binom{10}{5}$	$\Omega_{5,5}$	252
$\binom{10}{6}$	$\Omega_{6,4}$	210
$\binom{10}{7}$	$\Omega_{7,3}$	120
$\binom{10}{8}$	$\Omega_{8,2}$	45
$\binom{10}{9}$	$\Omega_{9,1}$	10
$\binom{10}{10}$	$\Omega_{10,0}$	1

Table 2.1: A table representing the number of states for each possible configuration of heads and tails in a system of 10 coins.

Note that if we add all the values in the third column, we get $\Omega_{\text{tot}} = 2^{10} = 1024$, which is the

total number of states. It immediately follows that the probability for each $\Omega_{k,10-k}$ is given by

$$P_{k,10-k} = \frac{\Omega_{k,10-k}}{\Omega_{\text{tot}}}. \quad (2.3)$$

Alternatively, one can rewrite the above expression in terms of entropy:

$$P_{k,10-k} = \frac{e^{S_{k,10-k}/k_B}}{\sum_{k=0}^{10} e^{S_{k,10-k}/k_B}}. \quad (2.4)$$

The probability of obtaining a particular state in the system is proportional to the exponential of the entropy: the higher the entropy, the more likely the state is to be observed. This is the essence of the Maximum Entropy Principle. Therefore, when we create models to analyze data, aiming for a Maximum Entropy approach is the safest bet: it gives us the most unbiased estimate factoring in certain constraints of the system, allowing us to avoid introducing assumptions that are not backed by data.

2.2 Shannon-Flavored Entropy

In 1948, Claude Shannon, an American mathematician known as the “father of information theory”, published a groundbreaking paper titled *A Mathematical Theory of Communication*. In it, he discussed the theory of data communications systems composed of three distinct elements: a source of data, a communication or compression channel, and a data receiver. In this discussion, he proved the **source coding theorem**, which states that the amount of information needed to losslessly compress a source of data is never less than the Shannon entropy of the source [5], which I will discuss in further detail below. If the source, assumed to be a random variable X , has Shannon entropy $H(X)$, then one needs at least $H(X)$ units of information to compress the source without any loss of information.

Shannon’s namesake entropy is defined as

$$H(X) = - \sum_{x \in \mathcal{X}} p(x) \log p(x), \quad (2.5)$$

where $H(X)$ is the Shannon entropy of a random variable X in the configuration space set $\mathcal{X}$

with probability distribution $p(x)$, which can be interpreted as the information associated with a random variable X and probability distribution $p(x)$. The logarithm term $\log p(x)$ has a base of e for our context, but it can also be in any other base, although base 2 and 10 are most common¹.

To make some more sense of the equation, let us break it down bit by bit. Recall that for an arbitrary quantity Q that depends on some x within a set $\mathcal{X}$, the expected value of Q is given by

$$\mathbb{E}[Q] = \sum_{x \in \mathcal{X}} p(x)Q(x). \quad (2.6)$$

With this in mind, the Shannon entropy can be re-interpreted as $\mathbb{E}[-\log p(x)]$. But what is $-\log p(x)$? It is a quantity called “self-information” or “surprisal”, which quantifies the amount of “surprise” one receives if they observe an event with probability $p(x)$. The more unlikely an event is, the more surprised one is when it occurs. Since probability distributions are normalized to 1 and must be non-negative, the negative sign in front of $\log p(x)$ is to ensure the result is positive, and therefore to ensure the Shannon entropy is positive.

Let us relate this back to our example in the previous section. Suppose we wanted to calculate the Shannon entropy of the system of coins. Modifying Table 2.1, we can calculate the Shannon entropy of this system, as shown in Table 2.2.

If we perform the calculation using the rightmost two columns, we get $H(X) = 2.701$ bits, where our random variable X is the state of the ten coins. For a point of comparison, the amount of information, equivalently Shannon entropy, needed to encode the system of a fair coin flip is 1 bit.

Initially, it may seem unintuitive that the Shannon entropy for ten coins is not just 10 bits! After all, are we not just looking at ten coin flips that are supposedly independent from each other? It turns out that, while the ten coins are independent from each other, the larger scale state of the system depends on the state of each coin. To illustrate this in more specificity, if the last coin you flip lands heads up, your system macrostate might be four heads and six tails, but if it lands tails

¹The units of information have different names depending on which base is used. For base 2, we have bits, for base e , nats, and for base 10, bans.

# of heads / 10	$\Omega_{k,10-k}$	$P_{k,10-k}$	$-\log_2 P_{k,10-k}$
0	1	$9.728 \cdot 10^{-4}$	10.006
1	10	$9.728 \cdot 10^{-3}$	6.684
2	45	$4.377 \cdot 10^{-2}$	4.514
3	120	0.1167	3.099
4	210	0.2043	2.291
5	252	0.2451	2.028
6	210	0.2043	2.291
7	120	0.1167	3.099
8	45	$4.377 \cdot 10^{-2}$	4.514
9	10	$9.728 \cdot 10^{-3}$	6.684
10	1	$9.728 \cdot 10^{-4}$	10.006

Table 2.2: A table representing the number of states for each possible configuration of heads and tails in a system of 10 coins, with the associated probabilities and surprisals. Note that we are using base 2 here for the coins, so the units of information are bits.

up, such a macrostate would instead be three heads and seven tails. The Shannon entropy $H(X)$ seeks to calculate the entropy of the *total* system and not just the aggregate of each individual, and as such is typically lower than the sum of each element. Mathematically, Shannon entropy satisfies a property known as **subadditivity** [6], where

$$H(A_0, A_1, \dots) \leq \sum_i H(A_i). \quad (2.7)$$

2.3 The Kullback-Leibler Divergence

After establishing our understanding of Shannon entropy as a measure of information content, we now turn to an intimately related concept that will be crucial for our analysis: the Kullback-Leibler (KL) divergence. Introduced in 1951 by Solomon Kullback and Richard Leibler through their seminal paper *On Information and Sufficiency*, the KL divergence provides a measure of the difference between two probability distributions [7]. Specifically, it quantifies the information lost when using one probability distribution to approximate another. More intuitively, it can be thought of as a “dissimilarity” between two probability distributions. Sometimes, it is easy to think about it as a “statistical distance” between two distributions, but applying the notion of

distance raises a number of mathematical concerns (such as not satisfying the triangle inequality or symmetry), which are highlighted due to its nonlinear nature.

For two discrete probability distributions, P and Q , defined over the same space $\mathcal{X}$, the Kullback-Leibler divergence from Q to P is defined as:

$$D_{\text{KL}}(P||Q) \equiv \sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)}. \quad (2.8)$$

where the logarithm can again be taken in any base, though we use the natural logarithm in this thesis. For the example involving coins, though, base 2 is used due to its convenient application in such scenarios. The KL divergence is always non-negative (i.e. $D_{\text{KL}}(P||Q) \geq 0$) and vanishes if and only if the distributions P and Q are identical.

To better understand the connection between KL divergence and Shannon Entropy, we can actually rewrite the KL divergence in terms of Shannon entropy:

$$D_{\text{KL}}(P||Q) = - \sum_{x \in \mathcal{X}} P(x) \log Q(x) - \left(- \sum_{x \in \mathcal{X}} P(x) \log P(x) \right) = H(P | Q) - H(P), \quad (2.9)$$

where $H(P | Q) \equiv - \sum_{x \in \mathcal{X}} P(x) \log Q(x)$ is a quantity called the **cross-entropy** between P and Q , and $H(P)$ is the aforementioned Shannon entropy of P . This decomposition reveals that the KL divergence measures the additional information required to describe the outcomes of P when using Q as our model, rather than using P directly.

To make these concepts more concrete, consider a simple example of a biased coin flip. Suppose we have a biased coin that lands heads with probability 0.7 and tails with probability 0.3. This represents our true distribution $P_{\text{heads}} = 0.7, P_{\text{tails}} = 0.3$. Now imagine we attempt to model this using a fair coin assumption Q with $Q_{\text{heads}} = Q_{\text{tails}} = 0.5$. The KL divergence from Q to P would be:

$$\begin{aligned}
D_{\text{KL}}(P||Q) &= 0.7 \log_2 \frac{0.7}{0.5} + 0.3 \log_2 \frac{0.3}{0.5} && (D_{\text{KL}} \text{ calculation 1}) \\
&= 0.7 \log_2 1.4 + 0.3 \log_2 0.6 \\
&\approx 0.119 \text{ bits.}
\end{aligned}$$

This positive value quantifies the information lost when we use the fair coin model to approximate the biased coin's behavior. If we were to reverse the calculation and compute $D_{\text{KL}}(Q||P)$, we would get a different value, illustrating the asymmetry of the KL divergence:

$$\begin{aligned}
D_{\text{KL}}(Q||P) &= 0.5 \log_2 \frac{0.5}{0.7} + 0.5 \log_2 \frac{0.5}{0.3} && (D_{\text{KL}} \text{ calculation 2}) \\
&\approx 0.126 \text{ bits.}
\end{aligned}$$

In the context of statistical modeling, the KL divergence serves as a natural measure of the quality of our approximations. When we construct a model to describe some observed data, we are effectively proposing a probability distribution Q to approximate the true underlying distribution P . The KL divergence provides a principled way to quantify how well our model distribution Q captures the essential features of the true distribution P . This interpretation will be particularly relevant when we discuss the connection between maximum entropy and maximum likelihood estimation in the next chapter.

2.4 Bringing Entropy and Divergence Together

A key property of the KL divergence that makes it especially useful for our purposes is its behavior under constraints. When we seek a model distribution Q that minimizes $D_{\text{KL}}(P||Q)$ subject to certain moment constraints (such as matching observed means and correlations), the solution turns out to be precisely the maximum entropy distribution subject to those same constraints. This remarkable connection between KL divergence minimization and entropy maximization will provide the theoretical foundation for our modeling approach.

With this theoretical framework in place, we can now turn to the specific application of these principles to our neural system model. The signals we aim to analyze can be obtained by examining neuronal activation in vertebrate retinæ. Given a set of observations, our goal is to determine the probability distribution of such a system. More precisely, we seek to answer the question: "How likely is it for me to obtain a particular state $\mathbf{x} = [x_1, x_2, \dots, x_k]$, where $x_i \in \{0, 1\}$ and k is the number of neurons, given the observed samples?" To ensure our probability distribution encapsulates as much information as possible while avoiding unwarranted assumptions, we employ the maximum entropy principle subject to appropriate constraints.

We now state constraints on the desired probability distribution. In particular, the probability should be normalized, and the means and correlations of the system should be anchored to data. Mathematically, this translates to the following relations:

$$\sum_{\mathbf{x}} p(\mathbf{x}) = 1, \quad (2.10)$$

$$\sum_{\mathbf{x}} p(\mathbf{x}) x_i = \bar{x}_i, \quad (2.11)$$

$$\sum_{\mathbf{x}} p(\mathbf{x}) x_i x_j = \bar{x}_i \bar{x}_j, \quad (2.12)$$

where $\bar{x}_i$ and $\bar{x}_i \bar{x}_j$ are the empirically determined means and pairwise correlations of the system, respectively.

The goal now is to maximize H given these constraints. This is a constrained optimization problem, and we can solve it using the method of Lagrange multipliers. The actual calculation is saved for Appendix A, but after grinding through the mathematics, the probability distribution that maximizes the entropy of the system is given by

$$p(\mathbf{x}) = \frac{1}{Z} \exp \left(\sum_i \beta_i x_i + \sum_{i,j} \gamma_{ij} x_i x_j \right), \quad (2.13)$$

where the β_i and γ_{ij} are the Lagrange multipliers corresponding to the mean and pairwise correlation constraints, and Z is the normalization constant. Note that in this formulation, $\gamma_{ii} = 0$, since it is not meaningful to discuss the "self-correlation" of a neuron. To expound on the nor-

malization constant, Z is the partition function, represented by

$$Z = \sum_{\mathbf{x}} \exp \left(\sum_i \beta_i x_i + \sum_{i,j} \gamma_{ij} x_i x_j \right). \quad (2.14)$$

This distribution is not merely a convenient mathematical construct; it can be shown to be the unique solution that maximizes the entropy of the system while satisfying our empirical constraints. Moreover, this same distribution minimizes the Kullback-Leibler divergence between our model and the empirical distribution, providing a theoretical foundation for its optimality. The detailed calculation demonstrating this equivalence between maximum entropy and minimum KL divergence is presented in Appendix A.

I now present a brief note on the equivalence of Shannon entropy and thermodynamic entropy. In the 1870s, Willard Gibbs and Ludwig Boltzmann wrote the canonical entropy of statistical mechanics as

$$S = -k_B \sum_i p_i \log(p_i), \quad (2.15)$$

where k_B is the Boltzmann constant and p_i is the probability of a microstate indexed from a larger configuration space. Comparing to equation 2.5, we observe that the fundamental structure of both equations are the same. Such similarity allows us to draw a connection between the two entropies, and this is indeed the case: the thermodynamic entropy in equation 2.15 is a direct application of Shannon entropy to a physical system where the base of the logarithm is base e .

Now that we have our proposed probability distribution and an intuition of how it connects to thermodynamic entropy, the task becomes one of determining the Lagrange multipliers β_i and γ_{ij} that best fit our observed data. This parameter estimation challenge leads us naturally to our next chapter, where we will discuss the practical implementation of this theoretical framework.

Ultimately, we are aiming to use this paradigm to predict the behavior of a stimulus neuron given the ambient behavior of the other network or environmental neurons. Additionally, we can time-shift the environmental neuron data forwards and backwards and analyze the stimulus neuron's behavior to model memory (timeshifting environmental neurons backwards) and prediction (forwards).

Chapter 3

Constructing and Validating the Model

The process to achieving the final goal of predicting a neuron’s behavior given the environmental neuronal network and characterizing model behavior is a long one. In the succeeding paragraph, I give a short overview of the main pipeline to achieving the goal, and concepts mentioned will be elucidated in the following two chapters.

Utilizing the paradigm of Maximum Likelihood Estimation and optimization methods, I determine a model $\mathcal{J}$ matrix — further elaborated on further in this chapter — that should reproduce true firing propensities and pairwise correlations between neurons.

In the next chapter I use this $\mathcal{J}$ matrix to develop a *likelihood ratio* based on the environmental neurons, from which I derive a binary prediction for the stimulus neuron. This prediction is compared to what the actual stimulus neuron output is and the result is piped into a confusion matrix for model characterization purposes. Ideally, if the model were perfect, no errors in prediction would be made, and the confusion matrix would be completely diagonal. This also holds for the case where data is timeshifted, which physically can be interpreted as inferring the network’s ability to store memories and make predictions. While many terms may seem confusing now, they will make more sense as we progress throughout this chapter and the next.

As promised, before I explain the fine details of modeling using Maximum Entropy (MaxEnt), I will discuss a few foundational concepts that are used specifically in the modeling and characterization process.

3.1 Maximum Likelihood Estimation (MLE)

The data I am expecting to receive is as an array of binary vectors, where each vector represents an output from an *a priori* unknown probability distribution that assigns a value for each entry in the entire space of possible outputs. Remember that for a length k binary vector representing a population of k neurons, our space of possible outputs is of size 2^k . How does one go about determining the probability of each state?

I thankfully have somewhere to start. In the last chapter, particularly in equation 2.13, I demonstrated that our probability distribution under the maximum entropy principle satisfying the constraints 2.12 is

$$p(\mathbf{x}) = \frac{1}{Z} \exp \left(\sum_i \beta_i x_i + \sum_{i,j} \gamma_{ij} x_i x_j \right). \quad (3.1)$$

If each x_i were binary, as is in our case, I offer an alternative interpretation of the exponential: let $\beta_i \mapsto \mathcal{J}_{ii}$ and $\gamma_{ij} \mapsto \mathcal{J}_{ij}$ represent the diagonal and off-diagonal elements of a combined $\mathcal{J}$ matrix, respectively. The utility in doing this is to propose a more elegant and practical approach to a difficult optimization problem — I can now rewrite

$$\sum_i \beta_i x_i + \sum_{i,j} \gamma_{ij} x_i x_j \longrightarrow \mathbf{x}^\top \mathcal{J} \mathbf{x}, \quad (3.2)$$

where $\mathbf{x} = [\dots, x_i, \dots]$ represents our binary state vector and $\mathcal{J}$ contains all information about the propensities of each neuron to fire (the β_i s) and their pairwise correlations in the network (the γ_{ij} s). This is equivalent to the earlier description because $x_i^2 = x_i$ when $x_i \in \{0, 1\}$.

Putting this all together, I can rewrite the probability distribution in equation 2.13 as

$$p(\mathbf{x}) = \frac{e^{\mathbf{x}^\top \mathcal{J} \mathbf{x}}}{Z}, \quad \text{where} \quad (3.3)$$

$$Z = \sum_{\mathbf{x}} e^{\mathbf{x}^\top \mathcal{J} \mathbf{x}}, \quad (3.4)$$

and $\mathcal{J}$ is a $k \times k$ matrix describing the correlation terms between each neuron in the system and propensities for each neuron to fire. The probability distribution P is determined by $\mathcal{J}$. Observe

a simplifying insight: since the correlation between neuron i and neuron j should be the same as the correlation between neuron j and neuron i , $\mathcal{J}$ is a symmetric matrix. However, I am still left with $k(k+1)/2$ unknown entries in $\mathcal{J}$.

To best determine the unknown entries in $\mathcal{J}$, statisticians have developed a number of methods. The method I have chosen to use is known as **Maximum Likelihood Estimation** (MLE). Given some sample of observed data, MLE aims to maximize a likelihood function associated with the probability distribution parameter space. By maximizing, one essentially finds the parameter values that make the observed data most likely to have occurred.

3.1.1 Before I Dive In: Why MLE?

As mentioned above, there are numerous methods to estimate parameters given observed data. However, MLE stands out in three ways that make it particularly useful. First, MLE is **asymptotically unbiased**, which means that given increasing sample sizes, the estimated parameters' expected will have a net zero expected difference from the true parameters. For finite data sets, the bias goes as $n^{-1/2}$ for data with n samples. Secondly, it is **asymptotically consistent**, ensuring that as sample sizes increase, the estimated parameters converge to the true parameters. Finally, it is also **asymptotically efficient**, meaning that it has the lowest asymptotic mean squared error in parameter estimation compared to other unbiased estimators. This translates to an estimator with the smallest variance among all unbiased estimators, implying greatest precision in parameter estimation.

Finally, the technique of MLE is *mathematically equivalent* to minimizing the Kullback-Leibler divergence between our model's distribution and the true distribution. One can make sense of this by recalling that the maximum entropy model of a system aims to find the probability distribution that maximizes "surprisal", translating to the distribution that is least biased against any particular outcome, and thus the most likely distribution given observed data.

3.2 Testing the Model

Before applying the model to real-world neuron data, I wanted to first quantify its performance, specifically how well it converges to a test $\mathcal{J}$ matrix with respect to sample size. For the testing phase, $k = 7$, so I had a 28-dimensional optimization problem, since $7(7 + 1)/2 = 28$ via our discussion in Section 3.1.

To test the model, I wrote a script called `isingtest.py` that generated symmetric $\mathcal{J}$ matrices with random entries by sampling from a normal distribution with $(\mu, \sigma) = (0, 0.2)$. Subsequently, I generated 20 $\mathcal{J}$ matrices to aid in statistical analysis. From there, I used equation 3.3 to generate a corresponding probability distribution for each $\mathcal{J}$ matrix. To make sampling from this distribution easier, I then mapped the probability distribution to the unit interval using `np.cumsum`, where the method `np.cumsum` within the `numpy` library in Python cumulatively sums over elements of an input array and returns an output array of the same size. From there, I drew random values from a uniform distribution over the unit interval and converted each value to its corresponding state. Each state was then appended to a `.txt` file for analysis. For each $\mathcal{J}$ matrix, I generated seven `.txt` files, containing 10, 40, 160, 640, 2,560, 10,240, and 40,960 samples, respectively. The number of samples chosen was rather arbitrarily; however, I wanted to observe logarithmic behavior in the convergence of the model to the true $\mathcal{J}$ matrix.

3.2.1 The Model is Asymptotically Unbiased

To analyze the performance of the model, I wrote another script called `isingfit.py`, which reads in each `.txt` file and assigned each state an entry in a $2^7 = 128$ element array P . The number of occurrences of each state was counted for each `.txt` file and P was updated accordingly. I then constructed an initial $\mathcal{J}_0$ matrix whose entries were also randomly sampled from a normal distribution with $(\mu, \sigma) = (0, 0.2)$. Using the Python function `scipy.minimize`, I then sought to minimize the KL divergence between the model's probability distribution and the observed probability distribution. To this end, I used the L-BFGS-B method (Limited Memory Broyden-Fletcher-Goldfarb-Shanno), which is a quasi-Newtonian iterative method based on gradient descent that is particularly well-suited for high-dimensional optimization problems [8].

For testing purposes, and since the model does not know the true $\mathcal{J}$ matrix, I also wrote a script that took the **Frobenius norm** between the true $\mathcal{J}$ matrix and the model's $\mathcal{J}$ matrix upon each iteration, which would give us a good sense of the total mean squared error between the two matrices. The Frobenius norm is defined as

$$\|A\|_F \equiv \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}, \quad (3.5)$$

where A is an $m \times n$ matrix and a_{ij} is the entry in the i th row and j th column of A . The Frobenius norm is essentially the Euclidean norm of the matrix when it is flattened into a vector.

Applying this to our case, denoting our test matrix as $\overline{\mathcal{J}}$ and model matrix as $\mathcal{J}$, the Frobenius norm would take the form

$$\|\overline{\mathcal{J}} - \mathcal{J}\|_F = \sqrt{\sum_{(i,j)}^{(k,k)} |\overline{\mathcal{J}}_{ij} - \mathcal{J}_{ij}|^2}.$$

Supposing the model matrix $\mathcal{J}$ is a good approximation of the true matrix $\overline{\mathcal{J}}$, I expect the Frobenius norm should converge to zero as the number of samples increases as promised by the asymptotically unbiased nature of the MLE method.

At the end of the testing process, I produced a graph that modeled the Frobenius norm against the number of samples, as seen in Figure 3.1.

The testing phase demonstrates that our maximum entropy model, implemented through Maximum Likelihood Estimation, successfully recovers the underlying correlation structure of simulated binary systems. The observed $n^{-1/2}$ convergence behavior aligns with theoretical expectations for MLE methods, providing confidence in the model's ability to extract meaningful patterns from binary data. This systematic validation using synthetic data sets of varying sizes establishes a robust foundation for the method's reliability and helps us understand its limitations and optimal operating conditions.

The model's logarithmic convergence behavior provides practical guidance for data collection: while larger datasets improve accuracy, they do so with diminishing returns. This insight will

prove valuable when working with experimental neuronal recordings, where data collection often faces practical constraints.

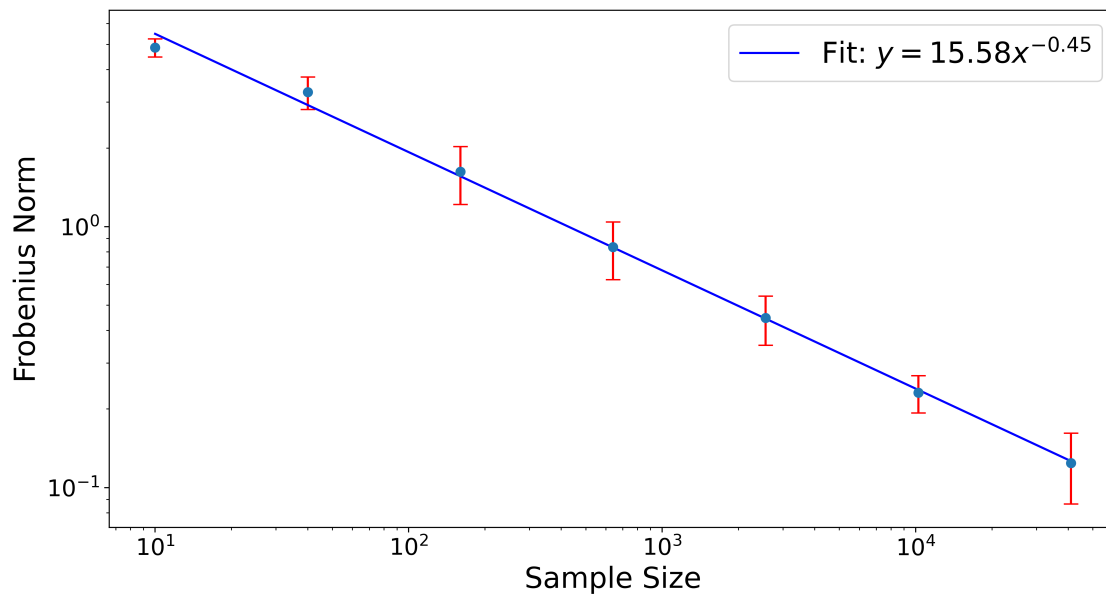


Figure 3.1: Frobenius norm of the difference between true and estimated $\mathcal{J}$ matrices as a function of sample size, shown in log-log scale. Error bars represent one standard deviation across 20 independently generated $\mathcal{J}$ matrices. The observed approximately $n^{-1/2}$ scaling (blue line) suggests the model's convergence follows the theoretical prediction for Maximum Likelihood Estimation.

Confusion Matrices: Classifying the Categorical

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ \vdots & & & & \end{pmatrix}$$

Figure 4.1: An example of test data used in Chapter 3. Note that the stimulus neuron is highlighted in red.

19

the true test parameters. The test data is a time series, with each row displaying the specific configuration of the neural system at a fixed time and each column displaying the time evolution of each neuron. Time steps between rows are assumed to be constant, and in the real data I analyzed, have steps of 100 ms. The 100 ms time steps are due to convention used in prior literature. I now spliced one of the columns from the rest of the data; this column is the stimulus neuron. Figure 4.1 provides a visualization of what such data looks like.

I spliced the data to produce the estimator for our neuronal system, hereon denoted $\hat{x}(s)$. The estimator is a function that uses a ratio of probabilities to yield a guess of whether the stimulus neuron is on or off. Splicing the data ensures that our model has no knowledge of the true state of our stimulus neuron and so its estimates cannot be biased. The estimator is based on the likelihood ratio, which, given an array corresponding to the environmental neuronal system at time t , yields a scalar describing how likely the stimulus neuron is active. This is mathematically derived as follows:

$$L(s) = \frac{p(x = 1 \mid s)}{p(x = 0 \mid s)} \quad (4.1)$$

$$= \frac{\exp \left(\begin{bmatrix} 1 & s^\top \end{bmatrix} \mathcal{J} \begin{bmatrix} 1 \\ s \end{bmatrix} \right) / Z}{\exp \left(\begin{bmatrix} 0 & s^\top \end{bmatrix} \mathcal{J} \begin{bmatrix} 0 \\ s \end{bmatrix} \right) / Z} \quad (4.2)$$

$$= \frac{\exp(J_{xx} + s^\top J_{sx} + J_{xs}s + s^\top J_{ss}s)}{\exp(s^\top J_{ss}s)} \quad (4.3)$$

$$= \exp(J_{xx} + s^\top J_{sx} + J_{xs}s) \quad (4.4)$$

I will now clarify some possibly unfamiliar notation. $p(x = 0 \mid s)$ describes the conditional probability that our stimulus neuron is *inactive* given a particular environmental neuron array ($x = 1$ implies activity). I have also broken $\mathcal{J}$ into block matrix representation via

$$\mathcal{J} = \begin{bmatrix} J_{xx} & J_{xs} \\ J_{sx} & J_{ss} \end{bmatrix}. \quad (4.5)$$

Armed with this interpretation, we can see that the likelihood ratio has two desirable properties: firstly, we do not need to calculate the partition function for each iteration, thus saving considerable computation time. Secondly, the likelihood ratio only relies on the stimulus neuron's propensity to fire and its pairwise correlations to environmental neurons, which is physically reassuring.

If $L(s) > 1$, the probability of our neuron being active and firing is greater than it not firing, so we predict $\hat{x}(s) = 1$. If $L(s) \leq 1$, we see the probability of our stimulus neuron firing is at most as great as it not firing, and thus $\hat{x}(s) = 0$.

Moving to characterize the quality of our estimator $\hat{x}$, I utilized a binary confusion matrix as is typically used for categorical classification schemes. I compared the model-estimated values using $\hat{x}(s)$ against empirical results $x(s)$. Instances corresponding to a correct prediction populated the diagonal entries of the confusion matrix, while incorrect predictions were in the off-diagonals.

I called this version of the confusion matrix "raw", since I later processed them to derive a "processed" confusion matrix, where each column summed to one. To give more insight into how I processed the "raw" confusion matrix to what is presented in Figure 4.2, I will utilize the following example.

Suppose I gather a dataset where the stimulus neuron, over the collection period of 720,000 bins, records being on (or a 1) 87,000 times and off (or a 0) 633,000 times. My predictor, using the MaxEnt model, correctly predicts the activated neuron 45,000 times and inactivated neuron 625,000 times, and therefore has 42,000 false negatives and 8,000 false positives. To process the "raw" confusion matrix into the values that appear in a confusion matrix like in Figure 4.2, I divide the left and right columns of the "raw" confusion matrix by their column sums — effectively normalizing each column — as highlighted in equation 4.6.

$$M_{\text{raw}} = \begin{bmatrix} 625,000 & 42,000 \\ 8,000 & 45,000 \end{bmatrix} \longrightarrow M_{\text{processed}} = \begin{bmatrix} \frac{625,000}{633,000} & \frac{42,000}{87,000} \\ \frac{8,000}{633,000} & \frac{45,000}{87,000} \end{bmatrix} = \begin{bmatrix} 0.9874 & 0.4828 \\ 0.0126 & 0.5172 \end{bmatrix} \quad (4.6)$$

Applying this to test data yielded promising preliminary verification. On the left subfigure within Figure 4.2, I ran a fit on a test on the empty correlation matrix, $\mathcal{J} = 0$. In this situation, all neurons are perfectly uncorrelated with each other, and no neuron has any propensity to fire. Therefore, behavior should be completely random, with about half of the stimulus neuron's time series data on and the other half off, simulating an unbiased coin flip.

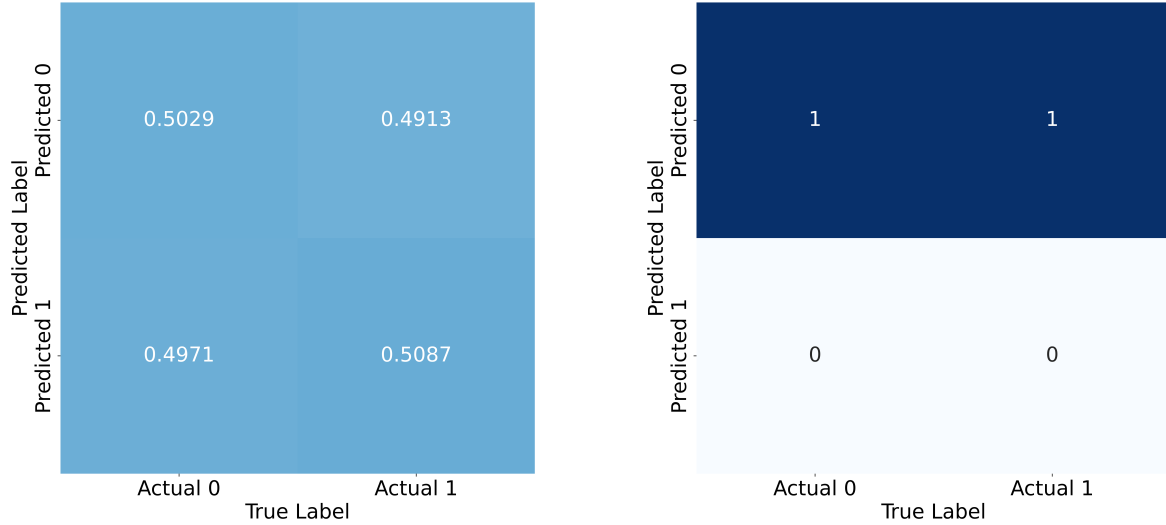


Figure 4.2: Confusion matrices generated given well-behaved data to ensure results were as expected by theory. **Left:** A confusion matrix corresponding to $\mathcal{J} = 0$, exhibiting expected behavior as all values are uncorrelated and should thus equally populate all entries of the matrix. **Right:** A confusion matrix corresponding to a diagonal $\mathcal{J}$ matrix with negative J_{xx} , again exhibiting expected behavior with the estimator unilaterally predicting $\hat{x}(s) = 0$. The scale on the right indicates what fraction of data carrying the true label was predicted as either on (1) or off (0). Both datasets contained 40,960 samples, and deeper blues correspond to a higher numerical value in the corresponding confusion matrix entry.

When the model fitted to the correlation matrix, it yielded values very close but not equal to 0

for all entries. As a result, the likelihood ratio stays very close to 1, yielding an even distribution in predicted 1s and 0s. Consequently, one would expect the confusion matrix to come out with 0.5s in all entries under perfect scenarios.

This is close to what is observed and residual effects can be explained by the fact that the fit did not converge identically to the zero matrix, yielding some small but nonzero value in J_{xx} which ultimately influenced the final distribution of values on the left subfigure in Figure 4.2. In essence, this is like an unbiased coin flip.

Another layer of verification was performed for a diagonal $\mathcal{J}$ matrix, where neurons had a bias in activation but no pairwise correlations were present, much like a biased coin flip. Since the likelihood ratio specifically depends on the propensity of our stimulus neuron firing and its correlations to the environmental neurons, we can easily adapt the likelihood to show that in this case, $L(s) = \exp(-J_{xx})$. Hence, when $J_{xx} > 0$, $L(s) < 1$, and thus our estimator must yield a 0 every time. Indeed, the right side of Figure 4.2 shows this happening, predicting no activity every instance.

Having established the theoretical framework and validated our methodology using synthetic (test) data, we are now positioned to extend this approach to real neuronal systems. Applying this on both timeshifting and non-timeshifted data will allow us to analyze how much information the stimulus neuron gives us about its ability to predict and store memories of environmental neuron states.

Chapter 5

What Happens in Real Neuron Populations?

5.1 Real Data: Cleaning and Processing, with some Modifications

The real data I received originated in a biology lab managed by Professor Joost le Feber and Professor Martina Lamberti (collaborators) from the University of Twente in the Netherlands as Matlab files. My collaborators recorded neuron activity on a multi-electrode array (MEA) with 60 sensors under differing conditions: local or focal stimulation, where “neuron 60” periodically stimulated the network, and global or light stimulation, where light was flashed at the entire neural network sporadically and succeeding network behavior was observed [10].

During each experiment, 20 hours of data were recorded, each data point only registering when a neuron fired, which typically left a signature, as seen in Figure 5.1a. Data was logged as a 2-tuple: the first reading was the sample number, which was converted to seconds assuming a 16 kHz sample rate, and the second reading an integer representing the index of the neuron that received the pulse. An example showing optogenetic — global or light-sensitive — stimulation activity over the course of about five minutes is shown in Figure 5.1b.

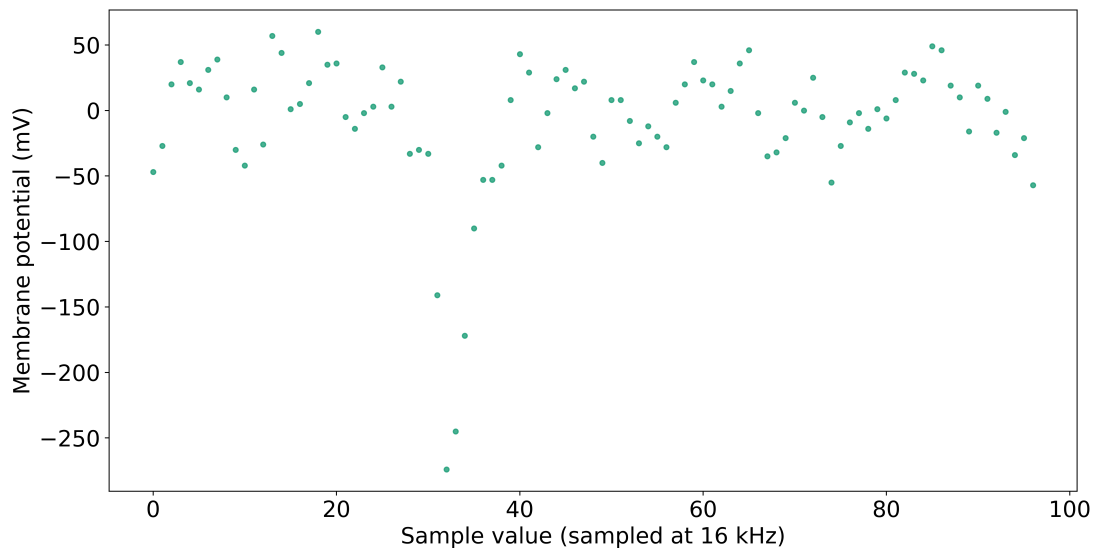
To adapt the data for use with the model I developed in previous chapters, I initially chose seven of the 60 neurons. Using all 60 neurons would result in a computational catastrophe: the

partition function would involve summing over 2^{60} states, and I would be left trying to optimize a 1,830-dimensional problem. Such a process would be extremely computationally expensive, so I attempted a practical workaround by using a subset of the neurons. I speculated that using seven of the original 60 neurons as a subset would capture enough information about the system without overwhelming the computational resources I had, so I went with that.

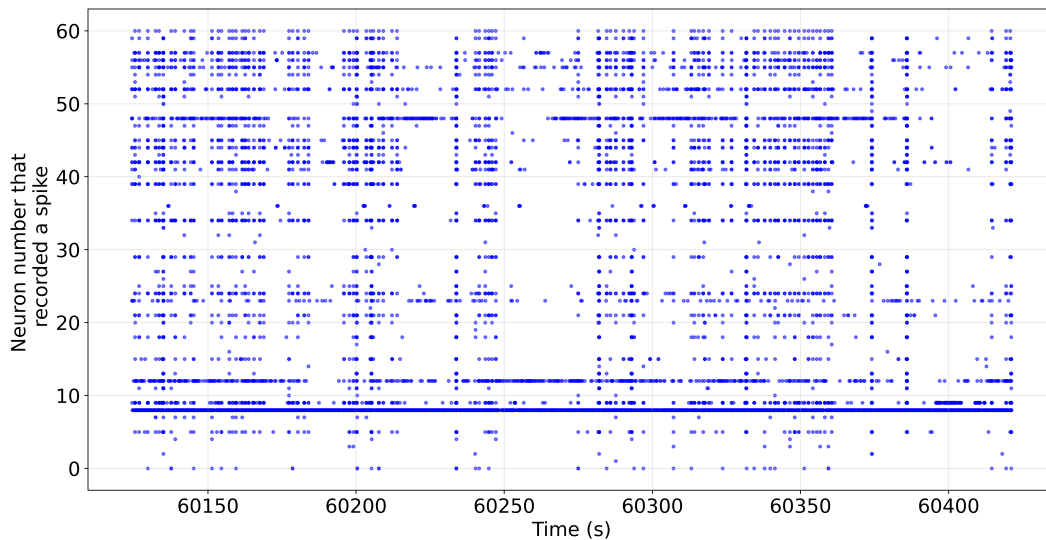
The subset of seven neurons was chosen somewhat randomly for the global stimulation; I inspected the neuron firing chart time series, like in Figure 5.1b, and chose what could be a representative sample based on which neurons fired synchronously. However, I acknowledge that there is work to be done in order to develop a more methodological approach — more discussion on this is saved for the further work chapter. I performed a similar method for local stimulation, except that “neuron 60” was always included: that was the stimulus neuron, responsible for pulsing electrical signals sporadically.

After subset selection and converting the time data to seconds, I further processed the data by binning the data into 100 ms partitions, which was outputted as a .txt file similar in form to what `isingfit.py` outputs. For instance, if I only collected two data points (1.234, 23) and (1.257, 47), and my subset was the neuron list (1, 12, 23, 36, 47, 49, 53), the resulting .txt file would contain the vector (0, 0, 1, 0, 1, 0, 0) in the 13th row, corresponding to all neuron pulses received between 1.2 and 1.3 seconds. The other rows would remain empty, as no neuron firings were collected otherwise. Now that the data was appropriately formatted, I proceeded with fitting the MaxEnt model and generating confusion matrices.

To understand how my model was performing, I wanted to compare its results to a baseline. The rationale for doing so was because most neurons are off for most of the time, so a reasonable baseline prediction would be to simulate an estimator predicting only 0s, physically interpreted as the stimulus neuron being perpetually off. A metric to quantify performance in this case is the predictive accuracy, which takes the total number of correct predictions and divides it by the total number of predictions made. Mathematically, one can think of this as a weighted trace of



(a) A scatter plot of a neuron's membrane potential as it fires as detected by the multi-electrode array. The dramatic drop in membrane potential around sample value 30 corresponds to the neuron firing; biologically, a neuron fires when its membrane potential reverses polarity and spikes briefly.



(b) A ~5-minute subset of the 20 hours of data from one of the global/optogenetic stimulation trials performed. One can observe that some neurons are more likely to fire in parallel.

the “raw” confusion matrix discussed in Chapter 4, and the specific mathematical formula is

$$A = \frac{\text{Tr}(M)}{\|M\|_1}, \quad (5.1)$$

where A is the predictive accuracy, M is the “raw” confusion matrix, and $\|\cdot\|_1$ denotes the matrix 1-norm, where one adds all entries of a matrix together.

5.2 An Initial Survey (and Rationalization) of the Data

I found that even choosing a subset of seven neurons — resulting in a 28-dimensional optimization problem — was still rather costly; each fit of the $\mathcal{J}$ matrix took two to three minutes. Therefore, I switched to an even smaller subset of five neurons, which improved computation time by a factor of about 8. The information loss was not significant: most confusion matrices generated with five neurons had almost identical predictive accuracy as those generated with seven neurons provided I judiciously chose my five neurons.

Additionally, I realized that unlike much of the test data on which I had validated my model, most of the neurons were off for most of the time, meaning the state vector $\mathbf{x} = \mathbf{0}$ was the most typical configuration. I was also initially surprised by the results of the fitted matrix since certain values were much smaller than I initially predicted, so I tried to rationalize it. For all $\mathcal{J}$ matrices, the diagonals were negative, and the correlation terms in J_{sx} (per the notation in equation 4.5) frequently exceeded 1. I initially believed that the off-diagonals in the $\mathcal{J}$ matrix directly corresponded to correlations between neurons, but it turns out that the correlation terms are actually nonlinearly related to the actual correlations between the neurons elucidated by Lamberti et al. [11]. For instance, my $\mathcal{J}$ matrix fitted on experiment 1 was

$$\mathcal{J} = \begin{bmatrix} -4.167 & 0.775 & 1.509 & 0.475 & -0.300 & 1.591 & -0.557 \\ 0.775 & -11.961 & -0.087 & 0.129 & -0.314 & -0.558 & 3.561 \\ 1.509 & -0.087 & -6.722 & -0.044 & -0.133 & 0.597 & 2.646 \\ 0.475 & 0.129 & -0.044 & -3.857 & 0.125 & 2.075 & -0.076 \\ -0.300 & -0.314 & -0.133 & 0.125 & -12.322 & 0.265 & 4.181 \\ 1.591 & -0.558 & 0.597 & 2.075 & 0.265 & -3.948 & -0.076 \\ -0.557 & 3.561 & 2.646 & -0.076 & 4.181 & -0.076 & -4.126 \end{bmatrix}. \quad (5.2)$$

In this particular experiment, the subset of neurons I chose yielded $P(\mathbf{x} = \mathbf{0}) = 0.87$, meaning all neurons were off for 87% of entries. Translated mathematically, this meant that

$$\mathbf{x}^\top \mathcal{J} \mathbf{x} = 0 \implies P(\mathbf{x} = \mathbf{0}) = \frac{e^{\mathbf{x}^\top \mathcal{J} \mathbf{x}}}{Z} = \frac{1}{Z} = 0.87. \quad (5.3)$$

Now that I had the probability for no neurons on, I wanted to calculate the probability when $\mathbf{x} = \mathbf{e}_i$, where $\mathbf{e}_i$ denoted a vector with only one neuron on at the i th position and the rest were off. This turned out to be

$$\mathbf{e}_i^\top \mathcal{J} \mathbf{e}_i = \mathcal{J}_{ii} \implies P(\mathbf{x} = \mathbf{e}_i) = \frac{\exp(\mathcal{J}_{ii})}{Z}. \quad (5.4)$$

Now, since the probability is normalized, none of these arguments can add up to more than 1, since by equation 5.3 the partition function itself is $Z = 1/0.87 \approx 1.15$. Therefore, $\mathcal{J}_{ii}$ is forced to be negative, explaining the diagonals.

I extended this further and found a relationship between elements of the $\mathcal{J}$ matrix and the probability of yielding a particular microstate of the system. Supposing a microstate has neurons $I = \{\alpha_1, \alpha_2, \dots\}$ on and the rest off, I denoted that particular state as $\mathbf{e}_{\alpha_1 \alpha_2 \dots}$ and found that

$$P(\mathbf{x} = \mathbf{e}_{\alpha_1 \alpha_2 \dots}) = \frac{\exp\left(\sum_{\alpha, \beta \in I, \alpha \neq \beta} (\mathcal{J}_{\alpha\alpha} + 2\mathcal{J}_{\alpha\beta})\right)}{Z}. \quad (5.5)$$

What this also meant is that, for k neurons, I had 2^k equations from which I could determine $\sim k^2$ unknowns: an overdetermined problem. However, thinking of this strictly mathematically was

not beneficial. Firstly, since the problem was overdetermined, a unique solution to the equations of constraint was not a guarantee. Furthermore, even if a unique solution was found, it would be biased towards the particular dataset, directly violating the maximum entropy principle of finding the $\mathcal{J}$ matrix providing the least bias given our empirical constraints, as mentioned in Chapter 2. With a marginally better idea of how the $\mathcal{J}$ matrices took their form, I continued numerically optimizing my matrices for both global and local stimulation. Further discussion of the results are discussed in the succeeding sections.

5.3 Global Stimulation

From my collaborators, I received a total of 10 experiments for global stimulation, where the cultured neurons were illuminated with light sporadically. Hence, there was no exact stimulus neuron, and therefore I chose an arbitrary neuron with which I built my predictor. Each set of data contained 20 hours of stimulation, corresponding to about 720,000 points total once partitioned into 100 ms bins.

After fitting suitable $\mathcal{J}$ matrices and generating their corresponding confusion matrices, I found that on average, an “off” neuron was accurately predicted 98.34% of the time, while an “on” neuron was accurately predicted 38.39% of the time. For the most part, the stimulus neuron was off, meaning there were significantly more bins where the true label of the stimulus neuron was 0. As a result, the overall predictive accuracy was typically around ~ 0.97 .

In comparison against the minimum probability flow (MPF) method, another model used to develop a predictor for stimulus-environment systems by minimizing a quantity related to the time derivative of the KL divergence[12], the MaxEnt model based on MLE fares marginally worse in predicting 0s given actual 0s, at 98.34% versus 100%, but fares significantly better at predicting 1s given actual 1s, at 38.39% over 8.46%. Results using the MPF method were taken from a paper I am currently writing with Professor Marzen and numerous other 5C students.

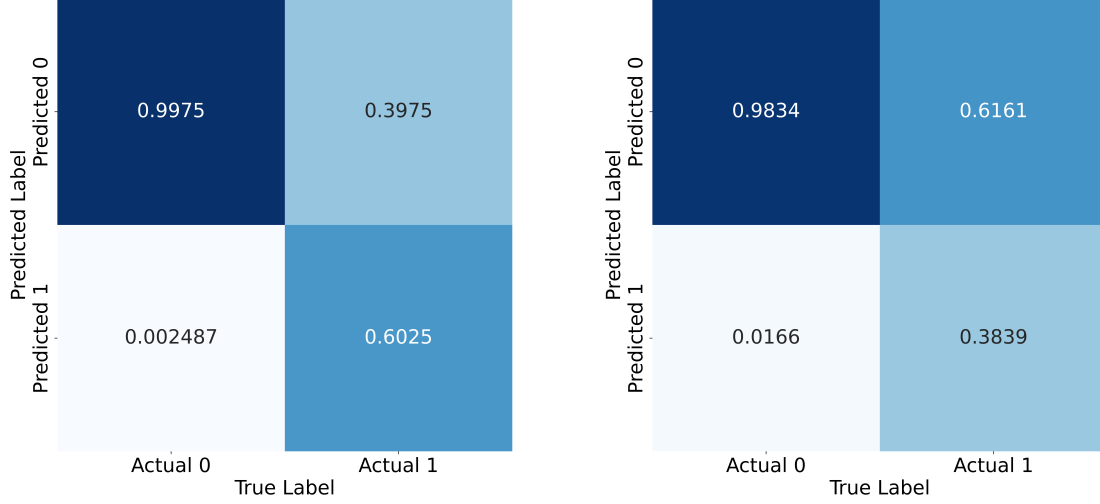


Figure 5.2: Two confusion matrices for global stimulation without timeshift, generated by a five-neuron subset of the original 60. **Left:** my confusion matrix corresponding to the best obtained result using the MaxEnt model, with the predictor accurately guessing inactivity 99.75% of the time and activity 60.25% of the time. **Right:** my confusion matrix corresponding to the average results over all ten experiments. The 1σ variations are 0.0188 for the first column and 0.1634 for the second column. Deeper blues on the confusion matrices correspond to higher numerical values in the corresponding confusion matrix entry.

To contextualize my comparison against the baseline prediction, it is useful to note that the processed confusion matrix corresponding to the baseline prediction is given by

$$M_{\text{baseline}} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}. \quad (5.6)$$

As shown in Figure 5.3, the overall predictive accuracy when the timeshift is 0 ms is better than the baseline, with an average predictive accuracy of 0.9261 in comparison to the baseline's 0.9011, yielding a 2.77% improvement. This implies the model is able to store a more accurate copy of the environment at present. While 2.77% does not seem significant, note that the baseline prediction exclusively predicts 0s and does so correctly more than 90% of the time. Therefore, the observed marginal improvement actually corresponds to a significant increase in the model's ability to accurately predict 1s, a much harder task to achieve due to its sparsity in the dataset.

When the data is timeshifted, I noticed something peculiar. Typically, the overall predictive accuracy improves slightly when the timeshift is negative, and the predictive accuracy when exclusively considering 1s is significantly higher. What this means is that our neurons model short-term memory better than a null prediction when the stimulus neuron is activated. There is no trend in the forward direction, however: for the most part, the model predicts at the same predictive accuracy as the baseline of 0s. An exception is with experiment 5, where the predictive accuracy is 0.1% higher at a timeshift of 100 ms, possibly implying that for this particular experiment our choice of neurons lets us model prediction with better-than-baseline accuracy. See Figure 5.3 for the general trend in predictive accuracies as a function of timeshift.

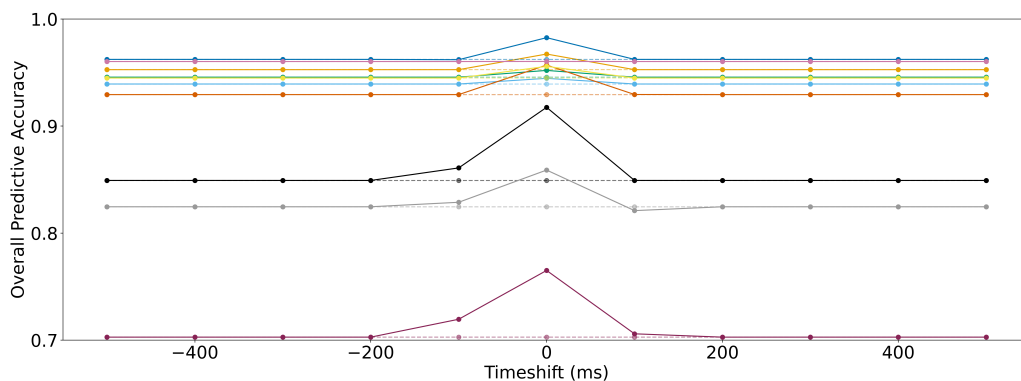


Figure 5.3: A plot of predictive accuracy as a function of timeshift for each of the ten experiments ran under global stimulation. Dashed, translucent lines indicate baseline predictive accuracies, while opaque, solid lines indicate predictive accuracies of the model. Each color represents one of the ten experiments received.

5.4 Local Stimulation

I received a total of 8 experiments for local stimulation, where one of the neurons, denoted “neuron 60”, acted as a source of local electrical stimulation. My goal with this subset of the project was to investigate how well I could predict the behavior of this stimulus neuron. As with the global stimulation, each dataset contained roughly 20 hours of stimulation, corresponding to about 720,000 points in 100 ms bins.

This time, because the neuron was localized, the effects are clear. The MaxEnt model, on average,

was much better at predicting when the neuron was active: 67.01% in comparison to 38.39% for global stimulation, as can be verified by comparing Figure 5.2 and Figure 5.4. In comparison to the MPF method, though, the MaxEnt model predictor was marginally worse in both predicting 0s and 1s: the MPF method successfully predicted all 0s and accurately predicted 1s 73.1% of the time. Therefore, the overall predictive accuracy was marginally worse than the MPF method.

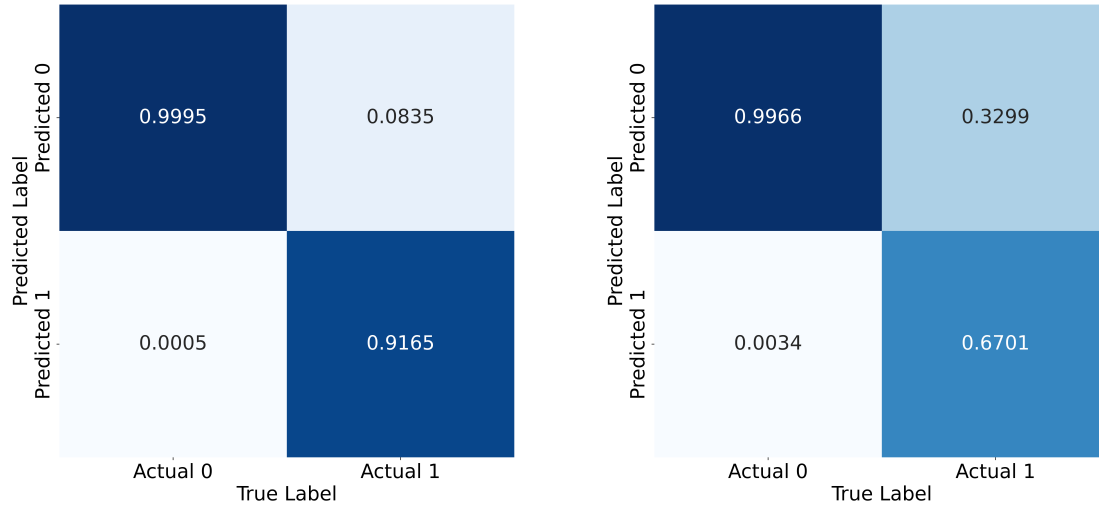


Figure 5.4: Two confusion matrices for local stimulation without timeshift, generated by a five-neuron subset of the original 60. **Left:** Confusion matrix corresponding to the best obtained result using the MaxEnt model, with the predictor accurately guessing inactivity 99.95% of the time and activity 91.65% of the time. **Right:** Confusion matrix corresponding to the average results over all eight experiments. The 1σ variations are 0.0025 for the first column and 0.2221 for the second column.

However, inspecting Figure 5.5, I observe that much like the case of global stimulation, when there is no timeshift (the neuron stores a copy of its present environment), the model does better than the baseline, with an average accuracy of 0.9896 as opposed to the baseline's 0.9785. I also notice that at a timeshift of -100 ms, the predictor, contrary to the global stimulation case, does slightly better than the baseline case. This implies that the predictor can store a better-than-baseline memory state, showing that our network of neurons are capable of storing memories with a capacity not exceeding 100 ms; higher timeshifts relegate the model to be only as good as baseline.

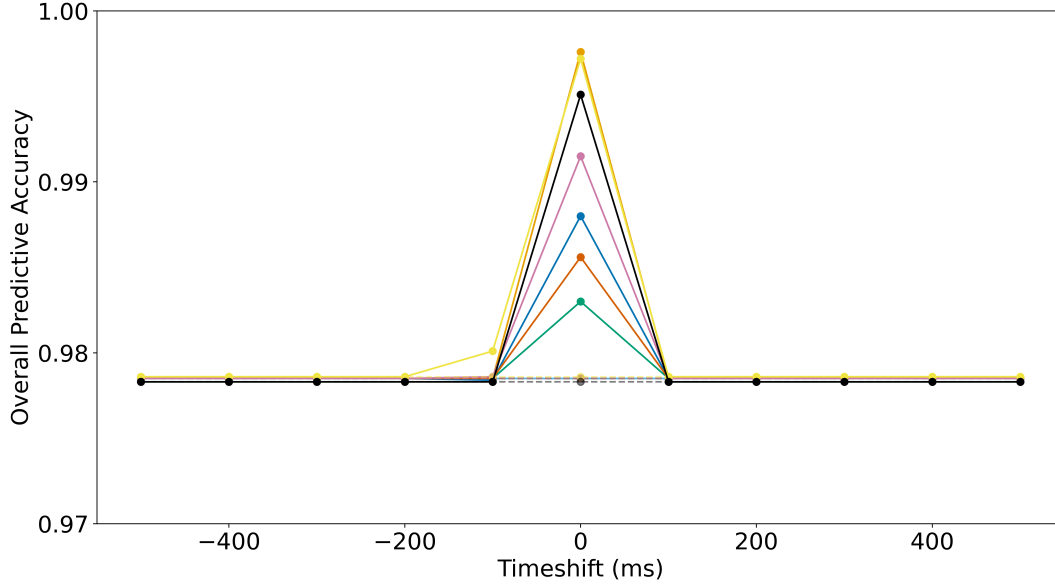


Figure 5.5: A plot of predictive accuracy as a function of timeshift for each of the eight experiments ran under local stimulation. Dashed, translucent lines indicate baseline predictive accuracies, while opaque, solid lines indicate predictive accuracies of the model. Different colors, like Figure 5.3, represent different experiments given to me.

Random neuron list	$\{6, 20, 37, 58, 60\}$	Random neuron list	$\{10, 50, 58, 59, 60\}$
Confusion matrix	$\begin{bmatrix} 0.9962 & 0.7141 \\ 0.0038 & 0.2859 \end{bmatrix}$	Confusion matrix	$\begin{bmatrix} 1.000 & 1.000 \\ 0.000 & 0.000 \end{bmatrix}$
Synchronized neuron list	$\{5, 20, 39, 54, 60\}$	Synchronized neuron list	$\{10, 26, 48, 56, 60\}$
Confusion matrix	$\begin{bmatrix} 0.9995 & 0.0834 \\ 0.0005 & 0.9166 \end{bmatrix}$	Confusion matrix	$\begin{bmatrix} 0.9949 & 0.7694 \\ 0.0051 & 0.2306 \end{bmatrix}$

Table 5.1: Two tables denoting the comparison between a randomized subset of neurons and a more organized subset of neurons based on inspection of figures such as Figure 5.1b for local stimulation. Experiment 9 is on the left table, experiment 12 is on the right.

It must be said that the choice of a subset is particularly important. In the local stimulation cases, I ran two trials of fitting my $\mathcal{J}$ matrix to experiments 9 and 12, each time varying the subset of neurons chosen. For one trial, each were selected with deliberation, chosen based approximately

on how well they synchronized with the stimulus neuron. The other trial was with a randomized subset of neurons. The results are shown in Table 5.1.

In both experiments, a more careful choosing of the subsets can yield a meaningful improvement in predictive accuracy. For example, in experiment 9, a careful choosing of the subset raises the predictive accuracy overall, improving the 0s predictive accuracy from 99.62% to 99.95% and 1s predictive accuracy from 28.59% to 91.66%. In another example, choosing a different subset of neurons in experiment 12, while declining in predictive accuracy for 0s to 99.49%, improves the 1s predictive accuracy from nothing to 23.06%. This emphasizes the importance of selecting the optimal subset of neurons to capture as much information about the system as possible while saving significant computational resources.

Chapter 6

Key Takeaways and Future Work

By applying the MaxEnt model and derived predictor on real neuronal data, I discovered that the model infers that the stimulus neuron in both the global and local stimulation cases can store a copy of its present environment through a better-than-baseline performance in both cases. Additionally, the model can vaguely infer memory propagating through the network, particularly for the case of local stimulation, with timeshifted neural data showing marginally better predictive accuracies under the MaxEnt model compared to baseline. Such inferences are limited to about 100 ms, though: timeshifts exceeding that threshold yields an estimator that is only as good as baseline, and the predictive accuracies speak for themselves, as shown in Figure 5.5. If the real neural networks do capture predictive capabilities of a future environmental state, the model does not capture it effectively. Most trials showed no improvement above baseline, with only a handful showing extremely marginal improvements in predictive accuracy.

Physically, both global and local stimulation occur without warning, making it impossible for the network to predict stimulus firing *a priori*. However, once triggered, the signal propagates through the network, leaving a memory imprint that the MaxEnt model captures. Consequently, the network encodes more information related to memory than to predictive capabilities.

As I continue to refine the model, my biggest hope is to develop an effective way to pick a subset of neurons that captures as much of the system's information and uses few computational resources, allowing for efficient and accurate optimization and prediction. As shown in Table

5.1, a well-chosen subset of neurons makes an impressive difference in predictive accuracies, so refining the practice would be massively beneficial. At present, I do not have a quantitative way of qualifying what counts as a “well-chosen” subset. Inspecting Figure 5.1b, I notice that a large wave of neurons fire occasionally, which I interpret to be a flash of light on the system. The same behavior occurs in the local stimulation case.

The MPF method discussed earlier was originally tried because the loss function it minimized required less computational resources than the full MaxEnt model implemented in this thesis, but my investigations thus far suggest that a strategically chosen group of neurons can potentially outperform the MPF model while maintaining efficiency. Prof. Marzen has suggested that Canonical Correlation Analysis (CCA) may aid in such programs. Treating our stimulus neuron as a one-component vector, X , of random variables and the network neurons as a 59-component vector, Y , of random variables, CCA may help find linear combinations of X and Y that maximize their correlation, revealing which neurons are most tightly correlated to our stimulus neuron [13] and ultimately improving the predictive accuracy of the MaxEnt model without sacrificing too many computational resources. This focused investigation into neural prediction reflects a microcosm of statistical mechanics’ broader power: achieving computational efficiency while preserving essential complexity. Such methodological refinements exemplify how statistical mechanics continues to evolve beyond its traditional domains, offering powerful frameworks for understanding emergent phenomena across increasingly diverse systems.

Statistical mechanics, first wielded to decode the thermal properties of matter and crucial to quantum theory’s birth, remains physics’ most elegant bridge between microscopic chaos and macroscopic order. Its revolutionary insight of emergent complexity from simpler parts interacting en masse has catapulted it into biophysics and beyond. What excites my imagination the most is the tantalizing possibility that the same mathematical framework that helped us understand simple gases might someday help decode consciousness itself, offering not just scientific insight but a profound new perspective on what it means to be human — an extraordinary arrangement of particles whose sentience emerges, like all things, from the fundamental principles that statistical mechanics first taught us to see.

Appendix A

Calculations for the Maximum Entropy Model

I will show that under certain constraints, the Maximum Entropy approach yields the Ising model probability distribution. Additionally, I will show that maximizing entropy minimizes the Kullback-Leibler divergence.

Recall that the Shannon entropy is

$$H[p(\mathbf{x})] = - \sum_{\mathbf{x}} p(\mathbf{x}) \log p(\mathbf{x}). \quad (\text{A.1})$$

For the model, denote $p(\mathbf{x}) \mapsto p_m(\mathbf{x})$. Next, we have the constraints

$$\sum_{\mathbf{x}} p_m(\mathbf{x}) = 1, \quad (\text{A.2})$$

$$\sum_{\mathbf{x}} x_i p_m(\mathbf{x}) = \bar{x}_i, \quad (\text{A.3})$$

$$\sum_{\mathbf{x}} x_i x_j p_m(\mathbf{x}) = \bar{x}_i \bar{x}_j, \quad (\text{A.4})$$

where the bars over the quantities represent experimentally derived means and correlations, respectively.

Constructing the Lagrangian for this system, we have

$$\mathcal{L}(p_m, \alpha, \beta_i, \gamma_{ij}) = H[p_m] + \alpha \left(\sum_{\mathbf{x}} p_m - 1 \right) + \sum_i \beta_i \left(\sum_{\mathbf{x}} x_i p_m - \bar{x}_i \right) + \sum_{i,j} \gamma_{ij} \left(\sum_{\mathbf{x}} x_i x_j p_m - \bar{x}_i \bar{x}_j \right). \quad (\text{A.5})$$

Set $\partial \mathcal{L} / \partial p_m[\mathbf{x}] = 0$, which is just

$$0 = - \sum_{\mathbf{x}'} (\log p_m + 1) + \alpha \left(\sum_{\mathbf{x}'} 1 \right) + \sum_i \beta_i \left(\sum_{\mathbf{x}'} x_i \right) + \sum_{i,j} \gamma_{ij} \left(\sum_{\mathbf{x}'} x_i x_j \right), \quad (\text{A.6})$$

where we actually select only the term where $\mathbf{x}' = \mathbf{x}$, simplifying the above to

$$0 = -(\log p_m + 1) + \alpha + \sum_i \beta_i x_i + \sum_{i,j} \gamma_{ij} x_i x_j. \quad (\text{A.7})$$

Solve for p_m in equation (A.7) to obtain

$$p_m = \exp \left((\alpha - 1) + \sum_i \beta_i x_i + \sum_{i,j} \gamma_{ij} x_i x_j \right) \quad (\text{A.8})$$

Utilizing constraint (A.2), we can write

$$1 = \sum_{\mathbf{x}} \exp \left((\alpha - 1) + \sum_i \beta_i x_i + \sum_{i,j} \gamma_{ij} x_i x_j \right) \quad (\text{A.9})$$

$$\implies \exp(-(\alpha - 1)) = \sum_{\mathbf{x}} \exp \left(\sum_i \beta_i x_i + \sum_{i,j} \gamma_{ij} x_i x_j \right) \quad (\text{A.10})$$

$$\equiv Z. \quad (\text{A.11})$$

The final probability distribution is thus

$$p_m = \frac{1}{Z} \exp \left(\sum_i \beta_i x_i + \sum_{i,j} \gamma_{ij} x_i x_j \right). \quad (\text{A.12})$$

Let us now minimize the Kullback-Leibler divergence. First, the KL divergence is given by

$$D_{\text{KL}}(P \mid Q) \equiv \sum_{\mathbf{x}} P \log \frac{P}{Q} = H[P \mid Q] - H[P], \quad (\text{A.13})$$

where the sum is over all possible states that the distributions can obtain. In this case, $P \mapsto p_d, Q \mapsto p_m$, where p_d is the empirically obtained probability distribution and p_m is the model's distribution. In general, we see that the gradient of D_{KL} is

$$\nabla D_{\text{KL}} = \begin{bmatrix} \frac{\partial}{\partial \beta_i} D_{\text{KL}} \\ \frac{\partial}{\partial \gamma_{ij}} D_{\text{KL}} \end{bmatrix} \quad (\text{A.14})$$

after substitution, because we get

$$D_{\text{KL}}(p_d \mid p_m) = \sum_{\mathbf{x}} p_d \left(\log p_d + \log Z - \left(\sum_i \beta_i x_i + \sum_{i,j} \gamma_{ij} x_i x_j \right) \right). \quad (\text{A.15})$$

Taking the first set of derivatives, observe

$$\frac{\partial}{\partial \beta_i} D_{\text{KL}} = \sum_{\mathbf{x}} p_d \left(\frac{\partial}{\partial \beta_i} \log Z - x_i \right) \quad (\text{A.16})$$

$$= \sum_{\mathbf{x}} p_d \left(\frac{1}{Z} \sum_{\mathbf{x}'} x_i \exp \left(\sum_i \beta_i x_i + \sum_{i,j} \gamma_{ij} x_i x_j \right) - x_i \right) \quad (\text{A.17})$$

$$= \underbrace{\sum_{\mathbf{x}} p_d \left(\sum_{\mathbf{x}'} x_i p_m \right)}_1 - \underbrace{\sum_{\mathbf{x}} p_d x_i}_{\bar{x}_i} \quad (\text{A.18})$$

$$= \sum_{\mathbf{x}'} x_i p_m - \bar{x}_i. \quad (\text{A.19})$$

Using similar reasoning, I find that

$$\frac{\partial}{\partial \gamma_{ij}} D_{\text{KL}} = \sum_{\mathbf{x}'} x_i x_j p_m - \bar{x}_i \bar{x}_j. \quad (\text{A.20})$$

Therefore, if these derivatives are set to zero, we retrieve our constraints,

$$\sum_{\mathbf{x}} x_i p_m = \overline{x_i}, \quad (\text{A.21})$$

$$\sum_{\mathbf{x}} x_i x_j p_m = \overline{x_i x_j}. \quad (\text{A.22})$$

As such, constraining the maximum entropy approach is equivalent to minimizing the Kullback-Leibler divergence of the model distribution and empirical distributions.

Appendix B

Non-Timeshifted, MaxEnt-inferred $\mathcal{J}$ Matrices for Experiments and Corresponding Confusion Matrices

This Appendix shows the inferred 5-by-5 $\mathcal{J}$ matrices and resulting confusion matrices for both the global and local stimulation trials in Chapter 5. Since the non-timeshifted data showed greatest improvements against baseline predictions, all $\mathcal{J}$ matrices and confusion matrices shown here are without timeshift. One can use these matrices to reconstruct an estimator and confusion matrices for each experiment, as highlighted in Chapter 4.

Additionally, for this Appendix, the following five-neuron subset was chosen for each experiment. Experiment numbers are repeated as they appeared in the dataset.

Global:

Experiment 1: (0, 11, 21, 50, 56)	Experiment 2: (26, 34, 37, 41, 58)	Experiment 3: (0, 1, 7, 10, 23)
Experiment 4: (0, 1, 7, 10, 23)	Experiment 5: (9, 11, 15, 52, 56)	Experiment 6: (12, 32, 45, 54, 58)
Experiment 7: (1, 5, 11, 28, 50)	Experiment 8: (2, 4, 11, 29, 54)	Experiment 9: (4, 12, 20, 27, 44)
Experiment 10: (5, 15, 25, 31, 41)		

Local:

Experiment 9: (5, 29, 39, 54, 60) Experiment 12: (10, 28, 46, 56, 60) Experiment 14: (5, 20, 23, 46, 60)
Experiment 15: (0, 20, 33, 50, 60) Experiment 16: (5, 13, 54, 57, 60) Experiment 17: (18, 35, 45, 56, 60)
Experiment 18: (12, 37, 53, 56, 60) Experiment 19: (13, 40, 50, 56, 60)

B.1 $\mathcal{J}$ Matrices For Global/Optogenetic Stimulation

Experiment # 1:

$$\mathcal{J}_1 = \begin{bmatrix} -3.870 & 1.480 & 0.894 & 0.009 & 0.023 \\ 1.480 & -3.421 & 1.319 & 2.114 & 0.831 \\ 0.894 & 1.319 & -4.805 & 0.429 & -0.299 \\ 0.009 & 2.114 & 0.429 & -5.890 & 0.012 \\ 0.023 & 0.831 & -0.299 & 0.012 & -4.037 \end{bmatrix} \quad (\text{B.1})$$

Experiment # 2:

$$\mathcal{J}_2 = \begin{bmatrix} -9.169 & 0.102 & 0.064 & -0.256 & 0.120 \\ 0.102 & -3.483 & -0.519 & -0.349 & 0.583 \\ 0.064 & -0.519 & -13.954 & -0.056 & -0.847 \\ -0.256 & -0.349 & -0.056 & -9.601 & 0.030 \\ 0.120 & 0.583 & -0.847 & 0.030 & -2.010 \end{bmatrix} \quad (\text{B.2})$$

Experiment # 3:

$$\mathcal{J}_3 = \begin{bmatrix} -3.281 & 0.945 & 0.620 & 0.278 & 0.849 \\ 0.945 & -2.655 & 0.622 & 0.235 & 0.701 \\ 0.620 & 0.622 & -7.795 & 0.499 & 0.693 \\ 0.278 & 0.235 & 0.499 & -4.042 & 0.454 \\ 0.849 & 0.701 & 0.693 & 0.454 & -4.702 \end{bmatrix} \quad (\text{B.3})$$

Experiment # 4:

$$\mathcal{J}_4 = \begin{bmatrix} -3.281 & 0.945 & 0.620 & 0.278 & 0.849 \\ 0.945 & -2.655 & 0.622 & 0.235 & 0.701 \\ 0.620 & 0.622 & -7.795 & 0.499 & 0.693 \\ 0.278 & 0.235 & 0.499 & -4.042 & 0.454 \\ 0.849 & 0.701 & 0.693 & 0.454 & -4.702 \end{bmatrix} \quad (\text{B.4})$$

Experiment # 5:

$$\mathcal{J}_5 = \begin{bmatrix} -5.368 & 0.731 & 2.136 & 0.783 & 0.757 \\ 0.731 & -7.057 & 1.882 & 0.706 & 0.460 \\ 2.136 & 1.882 & -4.136 & 1.623 & 0.935 \\ 0.783 & 0.706 & 1.623 & -8.026 & -0.398 \\ 0.757 & 0.460 & 0.935 & -0.398 & -7.881 \end{bmatrix} \quad (\text{B.5})$$

Experiment # 6:

$$\mathcal{J}_6 = \begin{bmatrix} -1.619 & 0.797 & 0.348 & 0.551 & 0.747 \\ 0.797 & -4.532 & 0.625 & 0.572 & 1.026 \\ 0.348 & 0.625 & -1.450 & 0.439 & 0.626 \\ 0.551 & 0.572 & 0.439 & -3.788 & 0.442 \\ 0.747 & 1.026 & 0.626 & 0.442 & -5.918 \end{bmatrix} \quad (\text{B.6})$$

Experiment # 7:

$$\mathcal{J}_7 = \begin{bmatrix} -10.564 & -0.167 & -0.128 & -0.097 & -0.335 \\ -0.167 & -11.922 & 0.153 & 0.073 & -0.643 \\ -0.128 & 0.153 & -8.327 & -0.235 & 1.727 \\ -0.097 & 0.073 & -0.235 & -10.776 & -0.462 \\ -0.335 & -0.643 & 1.727 & -0.462 & -2.376 \end{bmatrix} \quad (\text{B.7})$$

Experiment # 8:

$$\mathcal{J}_8 = \begin{bmatrix} -4.434 & 0.620 & 1.235 & 0.343 & 0.341 \\ 0.620 & -4.696 & 1.786 & 0.794 & 0.082 \\ 1.235 & 1.786 & -2.890 & 1.972 & 2.150 \\ 0.343 & 0.794 & 1.972 & -11.069 & 0.517 \\ 0.341 & 0.082 & 2.150 & 0.517 & -8.874 \end{bmatrix} \quad (\text{B.8})$$

Experiment # 9:

$$\mathcal{J}_{9,g} = \begin{bmatrix} -5.040 & 0.733 & 0.580 & 0.165 & 0.300 \\ 0.733 & -2.215 & 0.715 & 0.698 & 0.476 \\ 0.580 & 0.715 & -3.276 & 1.057 & 0.597 \\ 0.165 & 0.698 & 1.057 & -3.715 & 0.529 \\ 0.300 & 0.476 & 0.597 & 0.529 & -2.296 \end{bmatrix} \quad (\text{B.9})$$

Experiment # 10:

$$\mathcal{J}_{10} = \begin{bmatrix} -1.685 & 0.280 & 1.160 & 0.343 & 0.544 \\ 0.280 & -3.189 & 1.317 & 0.538 & 0.221 \\ 1.160 & 1.317 & -8.862 & 1.154 & 1.265 \\ 0.343 & 0.538 & 1.154 & -1.418 & 0.263 \\ 0.544 & 0.221 & 1.265 & 0.263 & -1.439 \end{bmatrix} \quad (\text{B.10})$$

B.2 $\mathcal{J}$ Matrices For Local/Focal stimulation

Experiment # 9:

$$\mathcal{J}_{9,l} = \begin{bmatrix} -7.296 & 4.342 & 1.211 & 0.082 & -0.546 \\ 4.342 & -9.603 & 1.751 & 0.073 & 0.144 \\ 1.211 & 1.751 & -2.284 & 0.903 & 1.130 \\ 0.082 & 0.073 & 0.903 & -1.713 & 1.233 \\ -0.546 & 0.144 & 1.130 & 1.233 & -5.522 \end{bmatrix} \quad (\text{B.11})$$

Experiment # 12:

$$\mathcal{J}_{12} = \begin{bmatrix} -4.984 & -0.238 & 0.765 & 0.815 & 1.143 \\ -0.238 & -5.268 & 1.113 & 0.563 & 0.595 \\ 0.765 & 1.113 & -3.400 & 0.988 & 0.978 \\ 0.815 & 0.563 & 0.988 & -3.937 & 0.828 \\ 1.143 & 0.595 & 0.978 & 0.828 & -3.597 \end{bmatrix} \quad (\text{B.12})$$

Experiment # 14:

$$\mathcal{J}_{14} = \begin{bmatrix} -6.397 & 1.703 & -0.632 & 1.395 & 0.548 \\ 1.703 & -3.685 & 1.008 & 0.274 & 1.250 \\ -0.632 & 1.008 & -4.221 & 0.671 & 0.493 \\ 1.395 & 0.274 & 0.671 & -3.103 & 0.442 \\ 0.548 & 1.250 & 0.493 & 0.442 & -2.474 \end{bmatrix} \quad (\text{B.13})$$

Experiment # 15:

$$\mathcal{J}_{15} = \begin{bmatrix} -6.425 & 0.441 & 0.397 & 1.852 & 1.153 \\ 0.441 & -4.786 & 1.221 & 1.594 & 1.214 \\ 0.397 & 1.221 & -6.170 & 1.560 & 0.711 \\ 1.852 & 1.594 & 1.560 & -3.861 & 1.319 \\ 1.153 & 1.214 & 0.711 & 1.319 & -5.911 \end{bmatrix} \quad (\text{B.14})$$

Experiment # 16:

$$\mathcal{J}_{16} = \begin{bmatrix} -6.030 & 0.920 & -1.381 & 1.638 & 2.069 \\ 0.920 & -4.962 & 1.358 & 0.351 & 1.383 \\ -1.381 & 1.358 & -6.629 & 1.362 & 1.844 \\ 1.638 & 0.351 & 1.362 & -6.203 & 1.772 \\ 2.069 & 1.383 & 1.844 & 1.772 & -4.134 \end{bmatrix} \quad (\text{B.15})$$

Experiment # 17:

$$\mathcal{J}_{17} = \begin{bmatrix} -6.728 & 1.090 & -0.263 & 0.722 & 3.326 \\ 1.090 & -7.599 & 0.077 & 0.953 & 1.497 \\ -0.263 & 0.077 & -7.825 & 2.351 & 1.014 \\ 0.722 & 0.953 & 2.351 & -4.525 & 2.023 \\ 3.326 & 1.497 & 1.014 & 2.023 & -5.597 \end{bmatrix} \quad (\text{B.16})$$

Experiment # 18:

$$\mathcal{J}_{18} = \begin{bmatrix} -7.683 & 1.326 & 0.767 & 1.286 & 4.675 \\ 1.326 & -5.536 & 1.683 & 2.234 & 0.189 \\ 0.767 & 1.683 & -7.944 & 1.737 & 0.015 \\ 1.286 & 2.234 & 1.737 & -3.560 & 0.465 \\ 4.675 & 0.189 & 0.015 & 0.465 & -8.545 \end{bmatrix} \quad (\text{B.17})$$

Experiment # 19:

$$\mathcal{J}_{19} = \begin{bmatrix} -7.273 & 2.292 & 0.302 & 0.636 & 4.267 \\ 2.292 & -6.501 & 1.545 & 1.409 & 0.278 \\ 0.302 & 1.545 & -3.292 & 0.933 & 0.135 \\ 0.636 & 1.409 & 0.933 & -2.761 & 0.122 \\ 4.267 & 0.278 & 0.135 & 0.122 & -8.146 \end{bmatrix} \quad (\text{B.18})$$

B.3 Confusion Matrices For Global/Optogenetic Stimulation

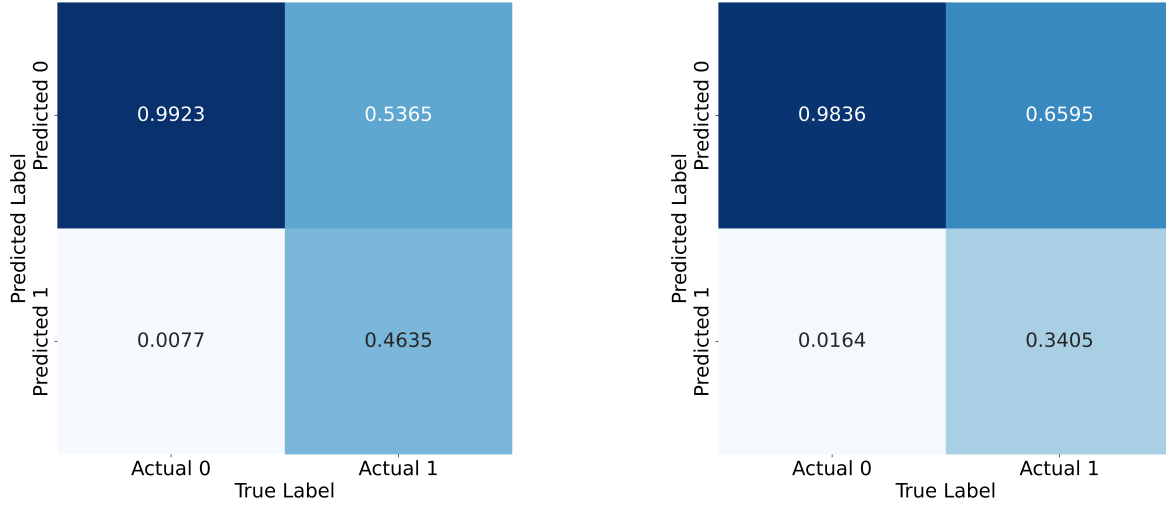


Figure B.1: Confusion matrices corresponding to the non-timeshifted data for experiments 1 and 2 using the inferred $\mathcal{J}$ matrix inferred above; experiment 1 is on the left, experiment 2 on the right.

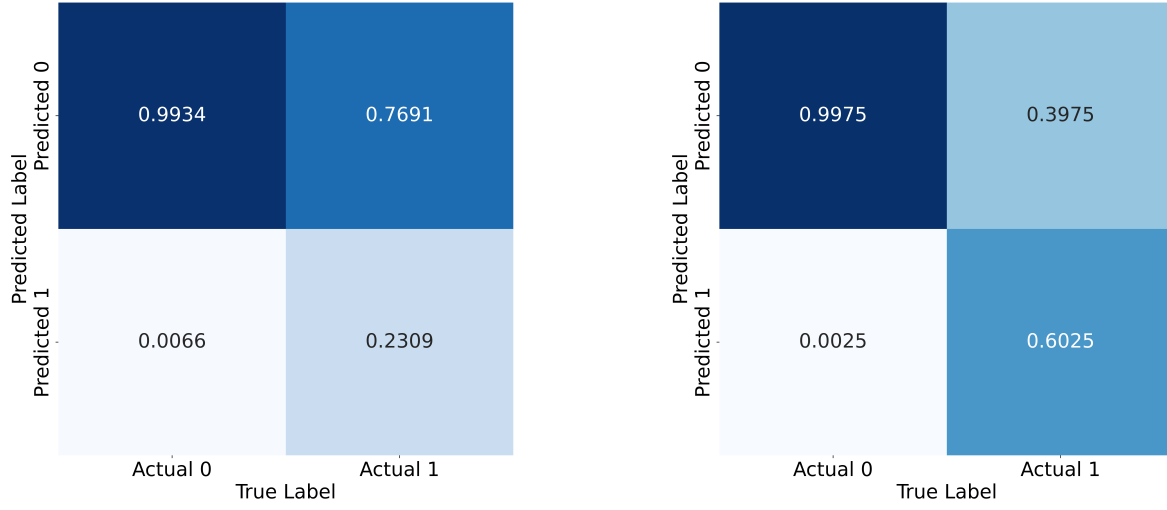


Figure B.2: Confusion matrices corresponding to the non-timeshifted data for experiments 3 and 4 using the inferred $\mathcal{J}$ matrix inferred above; experiment 3 is on the left, experiment 4 on the right.

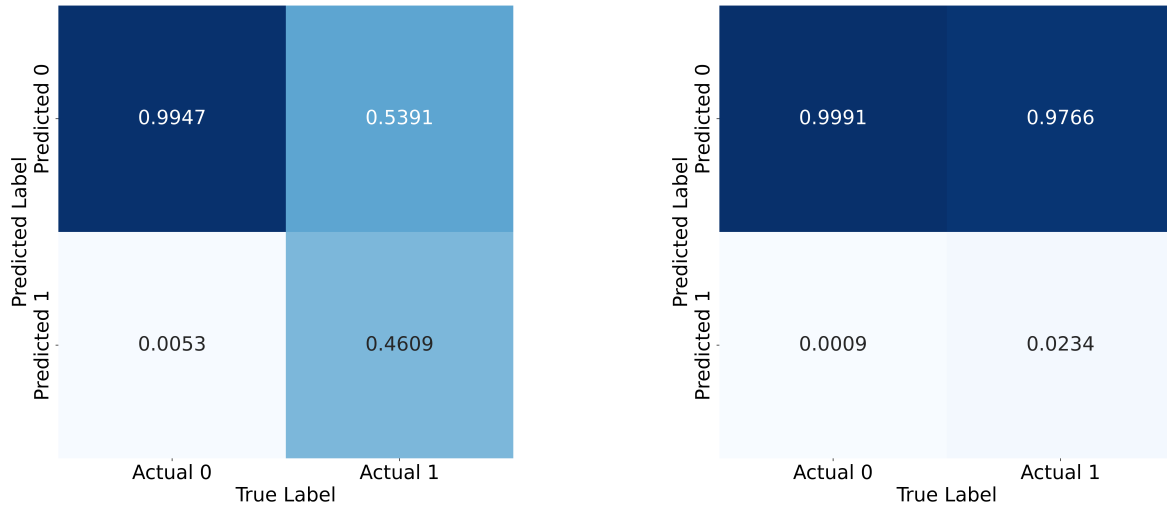


Figure B.3: Confusion matrices corresponding to the non-timeshifted data for experiments 5 and 6 using the inferred $\mathcal{J}$ matrix inferred above; experiment 5 is on the left, experiment 6 on the right.

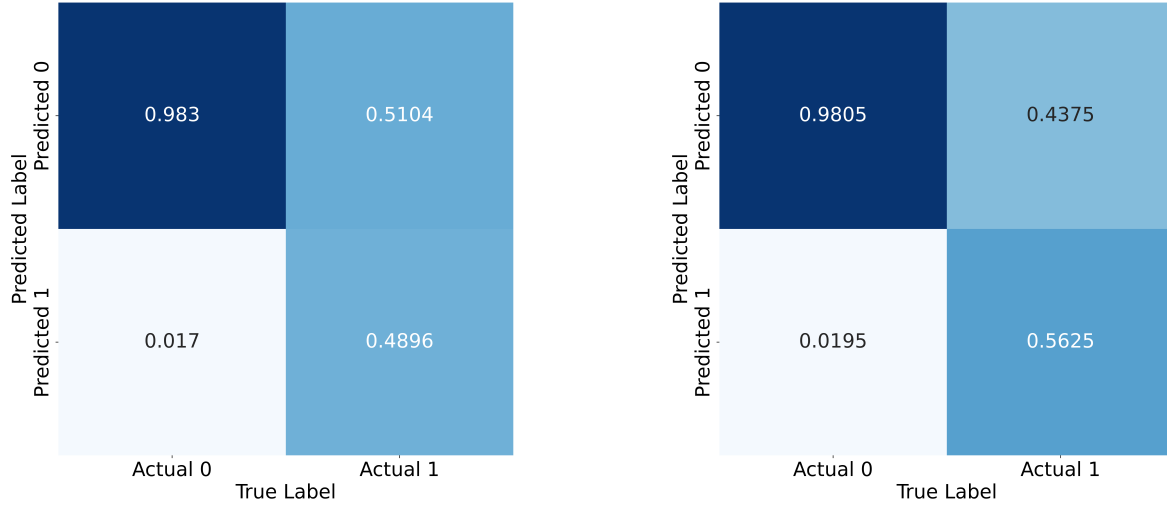


Figure B.4: Confusion matrices corresponding to the non-timeshifted data for experiments 7 and 8 using the inferred $\mathcal{J}$ matrix inferred above; experiment 7 is on the left, experiment 8 on the right.

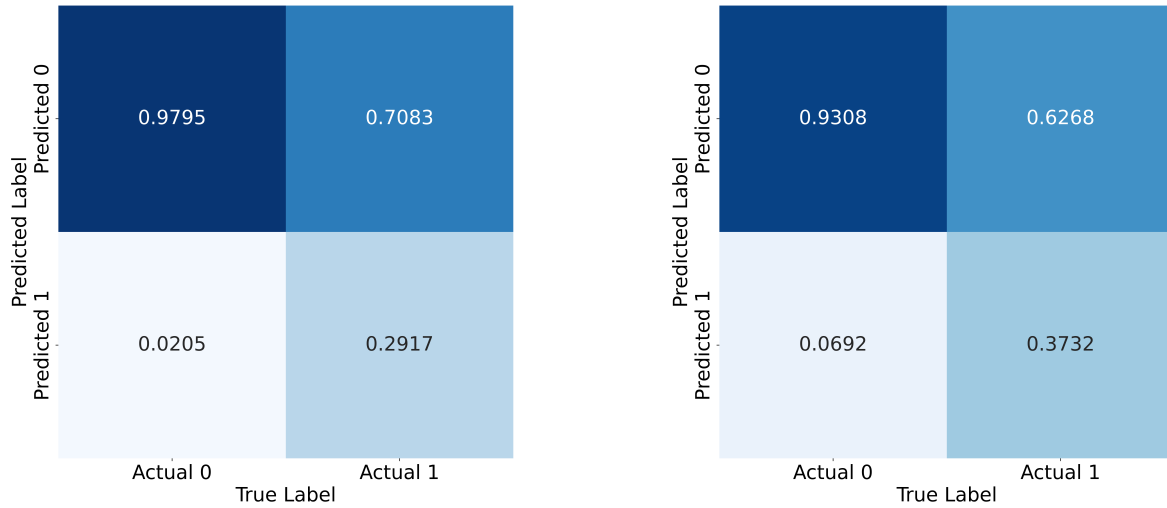


Figure B.5: Confusion matrices corresponding to the non-timeshifted data for experiments 9 and 10 using the inferred $\mathcal{J}$ matrix inferred above; experiment 9 is on the left, experiment 10 on the right.

B.4 Confusion Matrices For Local/Focal Stimulation

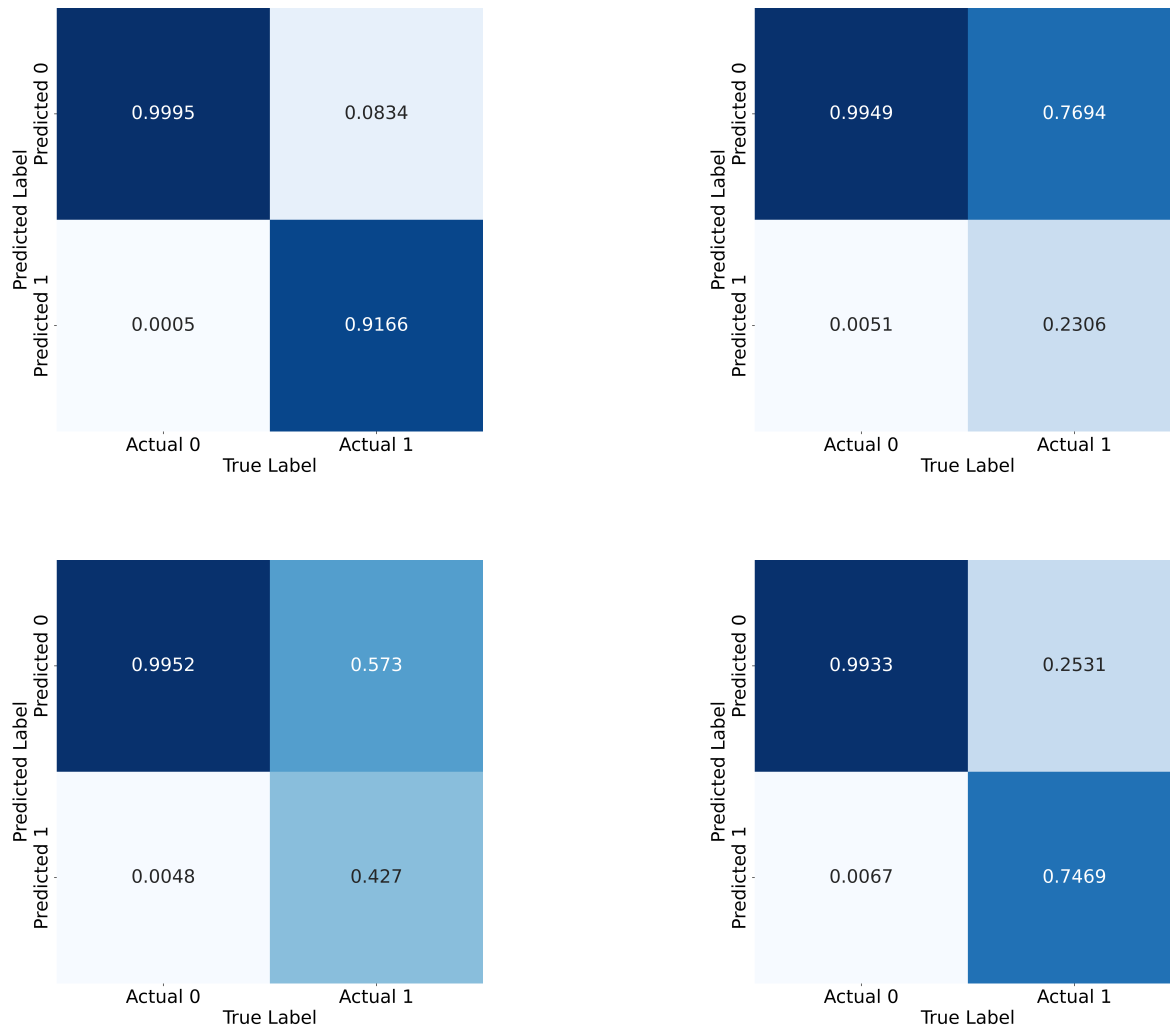


Figure B.6: Confusion matrices corresponding to the non-timeshifted data for experiments 9, 12, 14, and 15 using the inferred $\mathcal{J}$ matrices inferred above; experiment 9 is on the top left, experiment 12 is on the top right, experiment 14 is on the bottom left, and experiment 15 is on the bottom right.

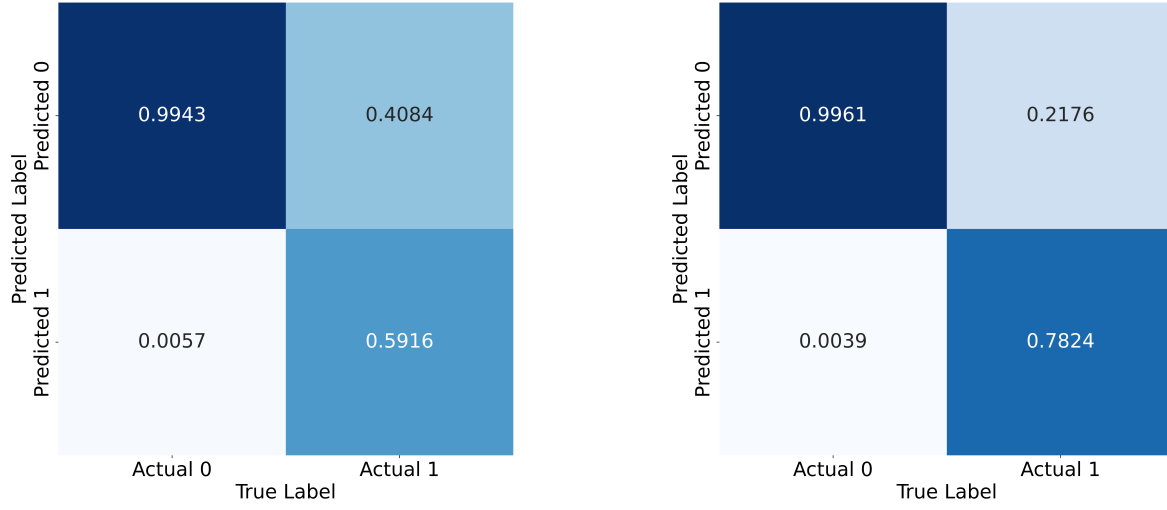


Figure B.7: Confusion matrices corresponding to the non-timeshifted data for experiments 16 and 17 using the inferred $\mathcal{J}$ matrix inferred above; experiment 16 is on the left, experiment 17 on the right.

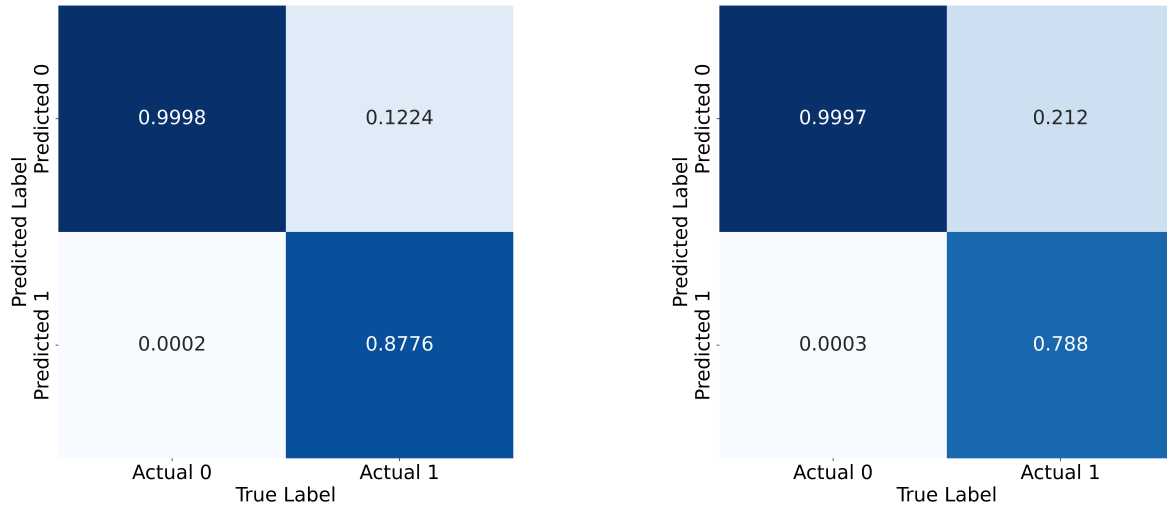


Figure B.8: Confusion matrices corresponding to the non-timeshifted data for experiments 18 and 19 using the inferred $\mathcal{J}$ matrix inferred above; experiment 18 is on the left, experiment 19 on the right.

Bibliography

- [1] E. T. Jaynes, The Physical Review **106**, 620 (1957).
- [2] E. T. Jaynes, The Physical Review **108**, 171 (1957).
- [3] E. Schneidman, M. J. Berry, R. Segev, and W. Bialek, Nature **440**, 1007 (2006), ISSN 0028-0836, 1476-4687.
- [4] G. Tkacik, E. Schneidman, M. J. Berry II, and W. Bialek, *Ising models for networks of real neurons* (2006), arXiv:q-bio.NC/0611072 Similar takeaways as the other Tkacik et al. paper.
- [5] C. E. Shannon, The Bell System Technical Journal **27**, 379 (1948).
- [6] E. Schneidman, S. Still, M. J. Berry, and W. Bialek, Physical Review Letters **91**, 238701 (2003), ISSN 0031-9007, 1079-7114.
- [7] S. Kullback and R. Liebler, The Annals of Mathematical Statistics **22**, 79 (1951).
- [8] D. C. Liu and J. Nocedal, Mathematical Programming **45**, 503 (1989), ISSN 0025-5610, 1436-4646.
- [9] S. V. Stehman, Remote Sensing of Environment **62**, 77 (1997), ISSN 00344257.
- [10] M. Lamberti, S. Tripathi, M. J. A. M. Van Putten, S. Marzen, and J. Le Feber, PNAS Nexus **2** (2023), ISSN 2752-6542.
- [11] M. Lamberti, M. Hess, I. Dias, M. Van Putten, J. Le Feber, and S. Marzen, Scientific Reports **12**, 9656 (2022), ISSN 2045-2322.

- [12] J. Sohl-Dickstein, P. B. Battaglino, and M. R. DeWeese, *Physical Review Letters* **107**, 220601 (2011), ISSN 0031-9007, 1079-7114.
- [13] *Canonical Correlation Analysis* (Springer Berlin Heidelberg, Berlin, Heidelberg, 2007), ISBN 978-3-540-72244-1.